

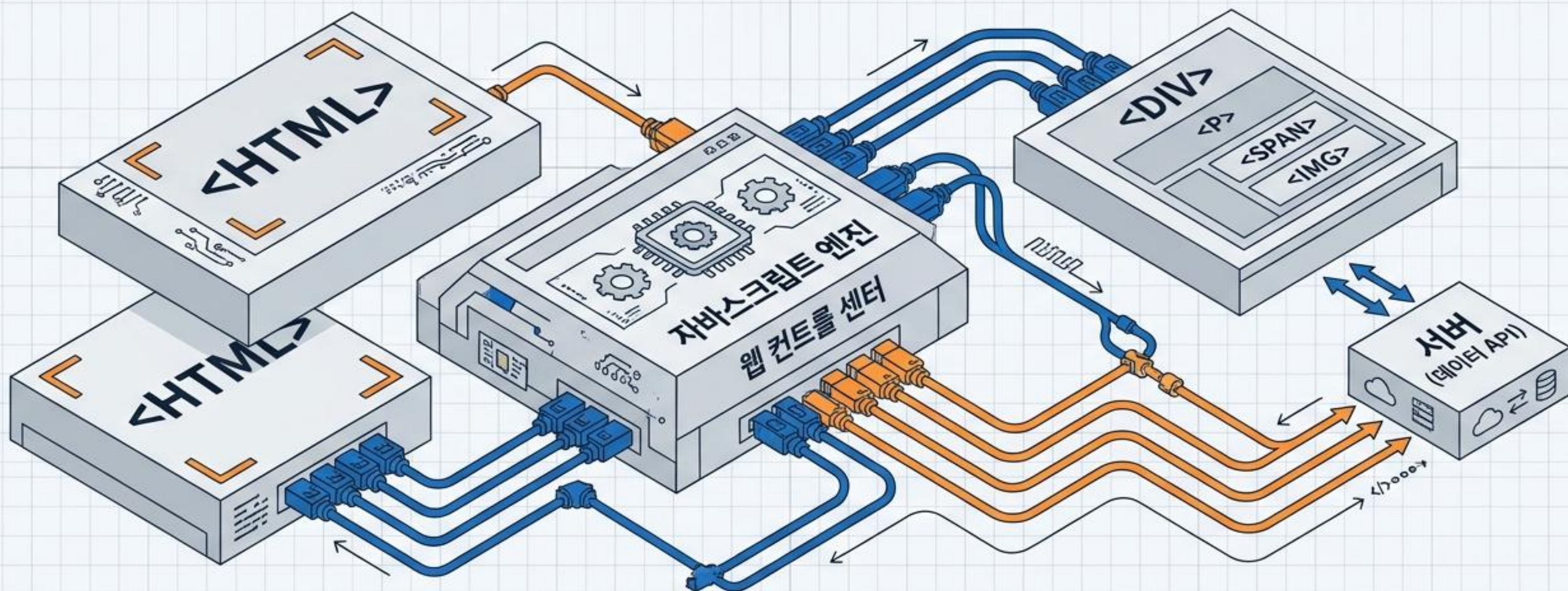
정적인 문서를 동적인 앱으로: 자바스크립트 웹 컨트롤 센터

태그 선택부터 비동기 통신(Ajax)까지, 인터랙티브 웹을 구축하는 4단계 엔진 설계도

[DOM 조작]

[이벤트 바인딩]

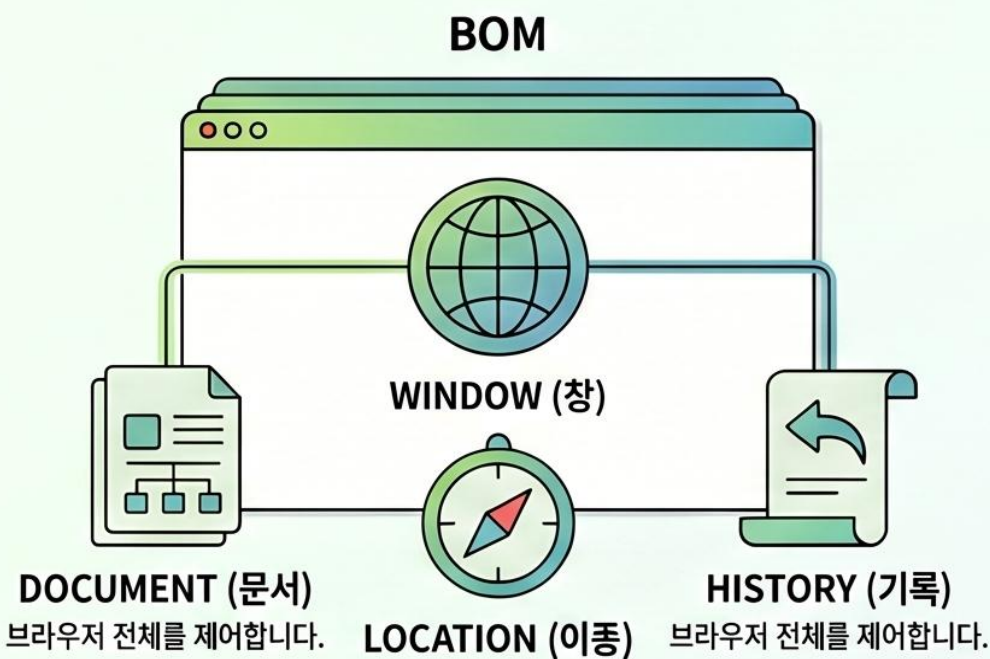
[비동기 Ajax]



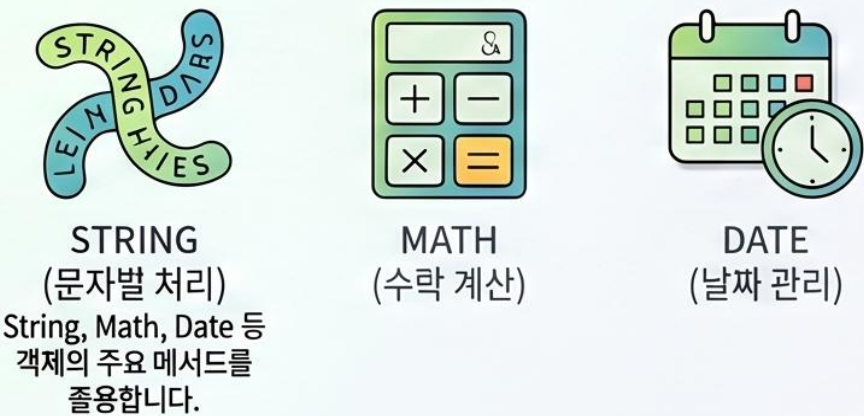
한눈에 보는 JavaScript & jQuery 핵심 요약 가이드

웹 페이지의 동적 제어를 위한 자바스크립트 내장 객체와 jQuery를 활용한 효율적인 DOM 조작법을 한눈에 파악하도록 도움.

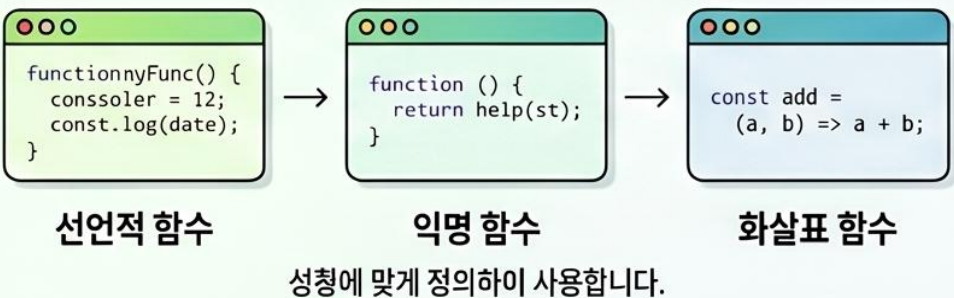
JavaScript 기초 및 내장 객체



핵심 내장 객체 활용



3가지 함수 정의 방식



순수 JavaScript와 jQuery의 문법 차이 비교

제어 대상	순수 JavaScript (Vanilla JS)	jQuery
요소 선택	document.querySelector('#id')	\$('#id')
값 설정	target.value = "값"	target.val("값")
스타일 변경	target.style.color = "red"	target.css("color", "red")

jQuery & DOM 문서 객체 제어

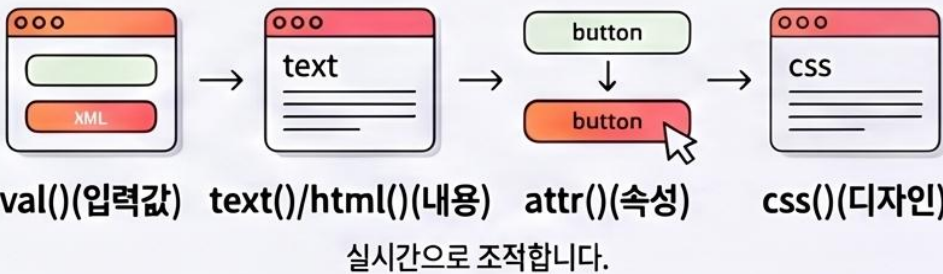


쉽고 빠른 요소 선택자 (Selector)

`$('#id')` `$('.class')` `$('#태그')`

자바스크립트보다 간용하게 태그를 선택합니다.

주요 태그 조작 메서드



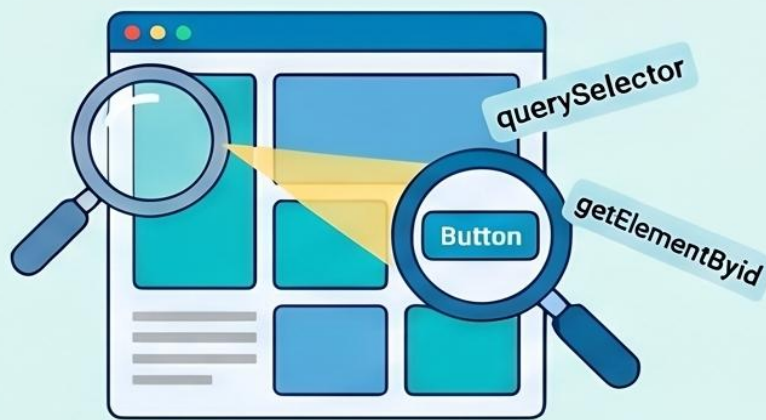
이벤트 리스너와 동적 효과



자바스크립트 & jQuery 핵심 요약 가이드

자바스크립트와 jQuery를 활용한 HTML 요소 제어, 이벤트 처리 및 Ajax 통신의 핵심 원리를 한눈에 파악하도록 도움.

DOM 조작 및 요소 선택



HTML 요소 선택 및 탐색

querySelector나 getElementById를 사용하여 ID, 클래스, 태그 기반으로 요소를 정확히 타겟팅합니다.



콘텐츠 및 스타일 실시간 변경

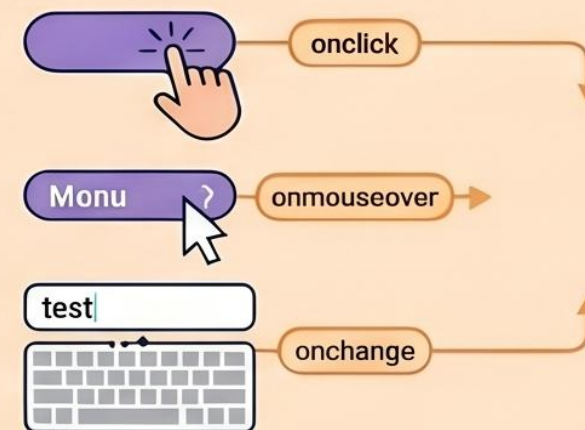
innerHTML로 내용을 교재하거나 .style 속성을 통해 배경색, 투명도 등 CSS를 등적으로 제어합니다.



속성(Attribute) 제어

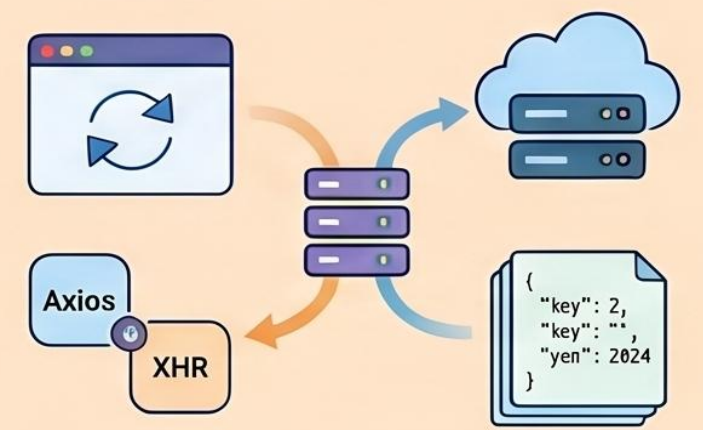
setAttribute 또는 .src와 같은 행버 변수 접근으로 이미지 경로 등 태그의 속성을 변경합니다.

이벤트 처리 및 비동기 통신



사용자 상호작용 이벤트

클릭(onclick), 마우스 오버(onmouseover), 값 변경(onchange) 등 다양한 사용자 동작을 감지하여 함수를 실행합니다.



Ajax를 이용한 비동기 데이터 요청

페이지 세로고침 없이 Axios나 XMLHttpRequest를 사용하여 서버에서 JSON 데이터를 읽어옵니다.

Before

```
const btn = document.querySelector("#btn");  
btn.addEventListener("click", () => {});
```

After

```
$('#btn').on('click', ...);
```

코드 효율성을 높이는 라이브러리 활용

복잡한 JS 코드를 JQuery의 \$(selector)나 \$.ajax를 통해 획기적으로 단축하여 구현 가능합니다.

한눈에 보는 jQuery 핵심 요약 가이드: "Write Less, Do More"

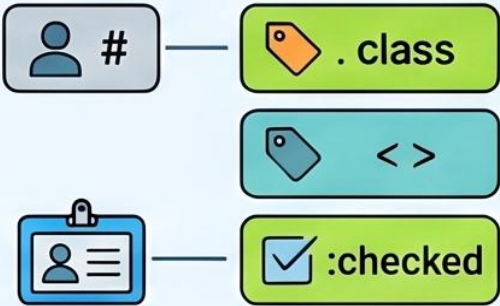
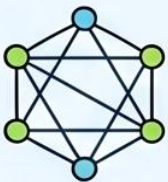
jQuery 기초 및 선택자 (Foundation)

 **핵심 문법:** `$(selector).action();` 

\$ 기호로 객체를 호출하고 선택자로 요소를 저장한 뒤 동작을 수행하는 직관적 구조입니다.

 **DOM Ready 축약형**
`$(function(){});`
HTML 문서 로딩이 완료된 시점에 코드가 실행되도록 보장하는 필수 안전장치입니다.

강력하고 다양한 요소 선택자:
ID(#), 클래스(.), 태그명은 물론 특정
순서(:first)나 상태(:checked)로
요소를 쉽게 찾습니다.



1 요소 선택 및 색상 변경

순수 JavaScript (Vanilla)

```
document.getElementById("id").style.color = "red";
```

2 내용 변경 차이

`.innerText`
(HTML 태그 해석 안 함)



3 요소 삭제 차이

`.removeChild()`
(복잡함)

jQuery

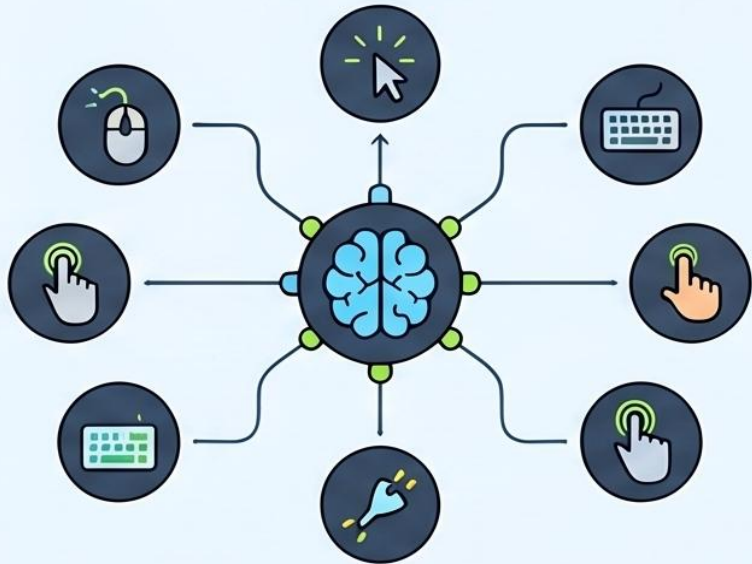
`$("#id").css("color", "red");`

`.text() / .html()`
(해석 여부 선택 가능)

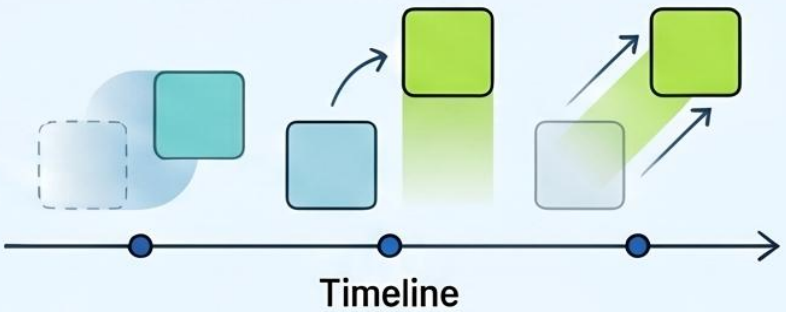
`.remove()` (전체 삭제)
`.empty()` (내부 비움)

실무 핵심 기능 (Core Actions)

이벤트 위임의 핵심
.on() 메서드
클릭, 마우스 오버 등 모든 이벤트를 처리하며, 동적으로 생성된 요소에도 이벤트를 바인딩합니다.



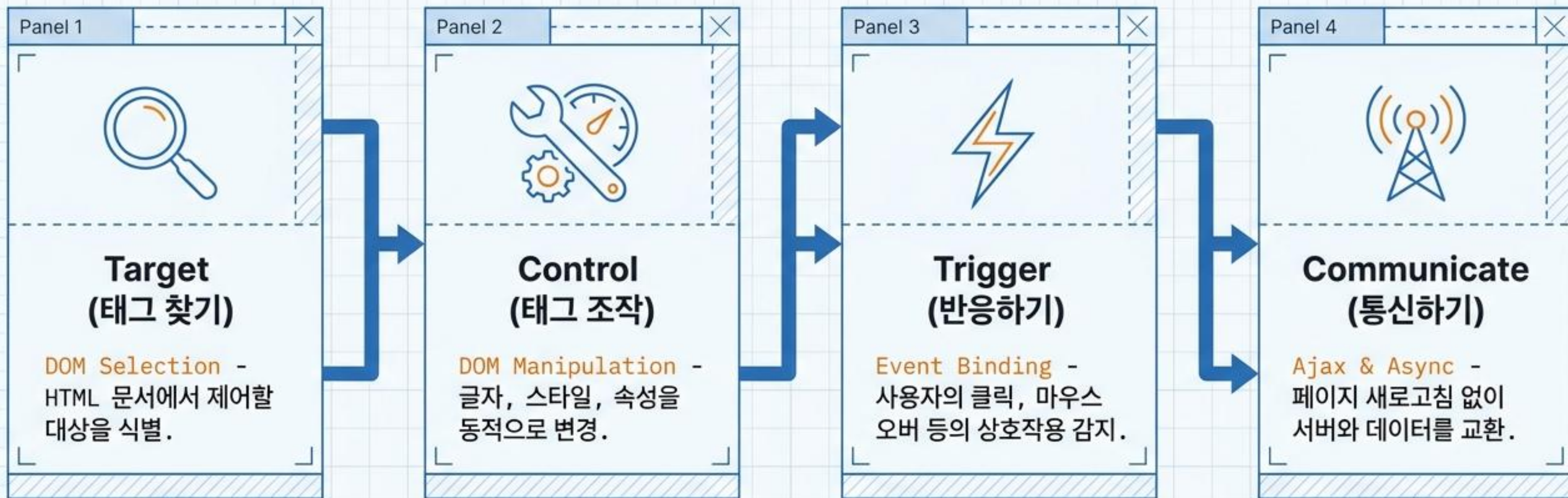
에니메이션 및 효과의 단순화
fadeIn, slideUp, animate 등 복잡한 시차 효과를 단 한 줄의 메서드로 구현합니다.



비동기 통신의 정석
\$.ajax()
페이지 새로고침 없이 서버와 데이터를 주고받는 측람한 과정을 설정 객체 하나로 통합합니다.

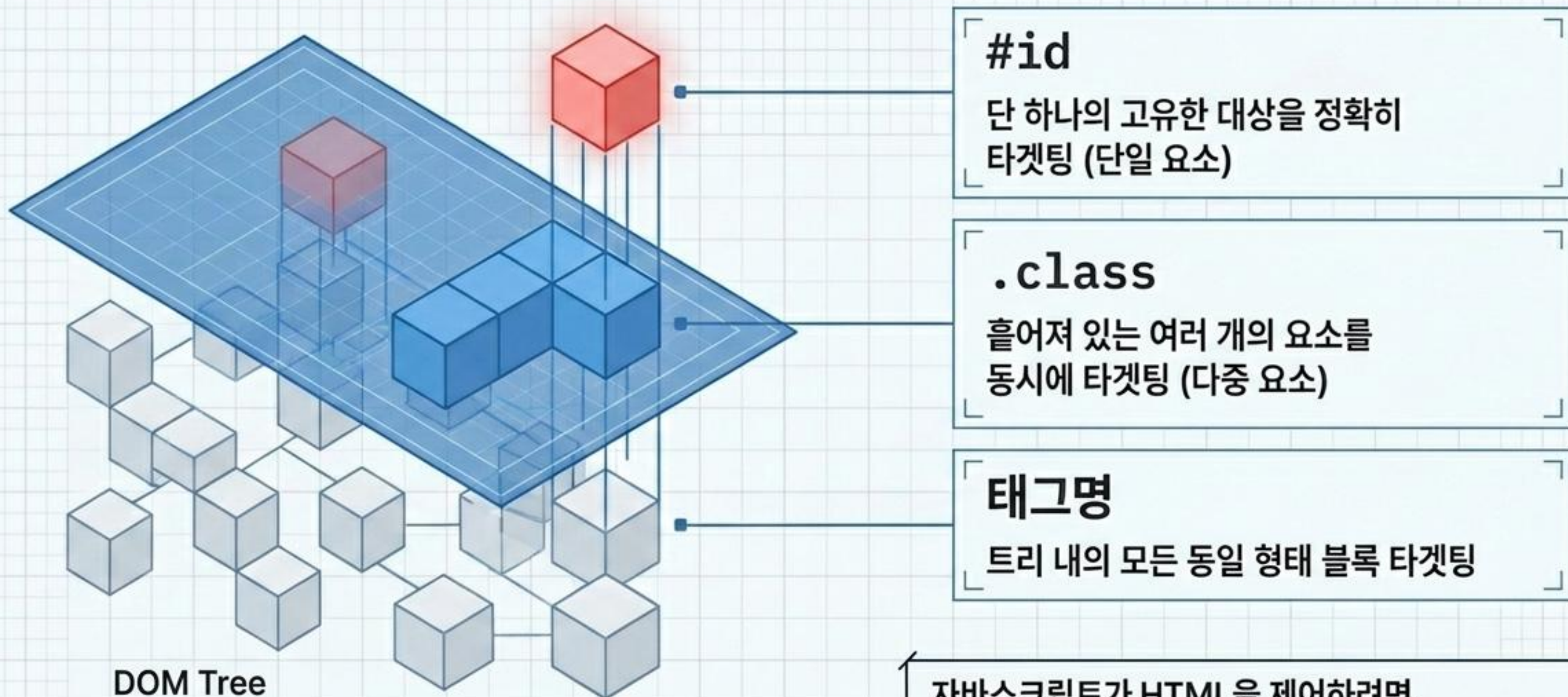


인터랙티브 웹을 구동하는 4단계 프로세스



이 청사진을 따라가며 각 모듈의 작동 원리를 해부해 봅니다.

Step 1: DOM 트리에 필터망 던지기



자바스크립트가 HTML을 제어하려면,
먼저 대상의 '주소(선택자)'를 정확히 찾아내야 합니다.

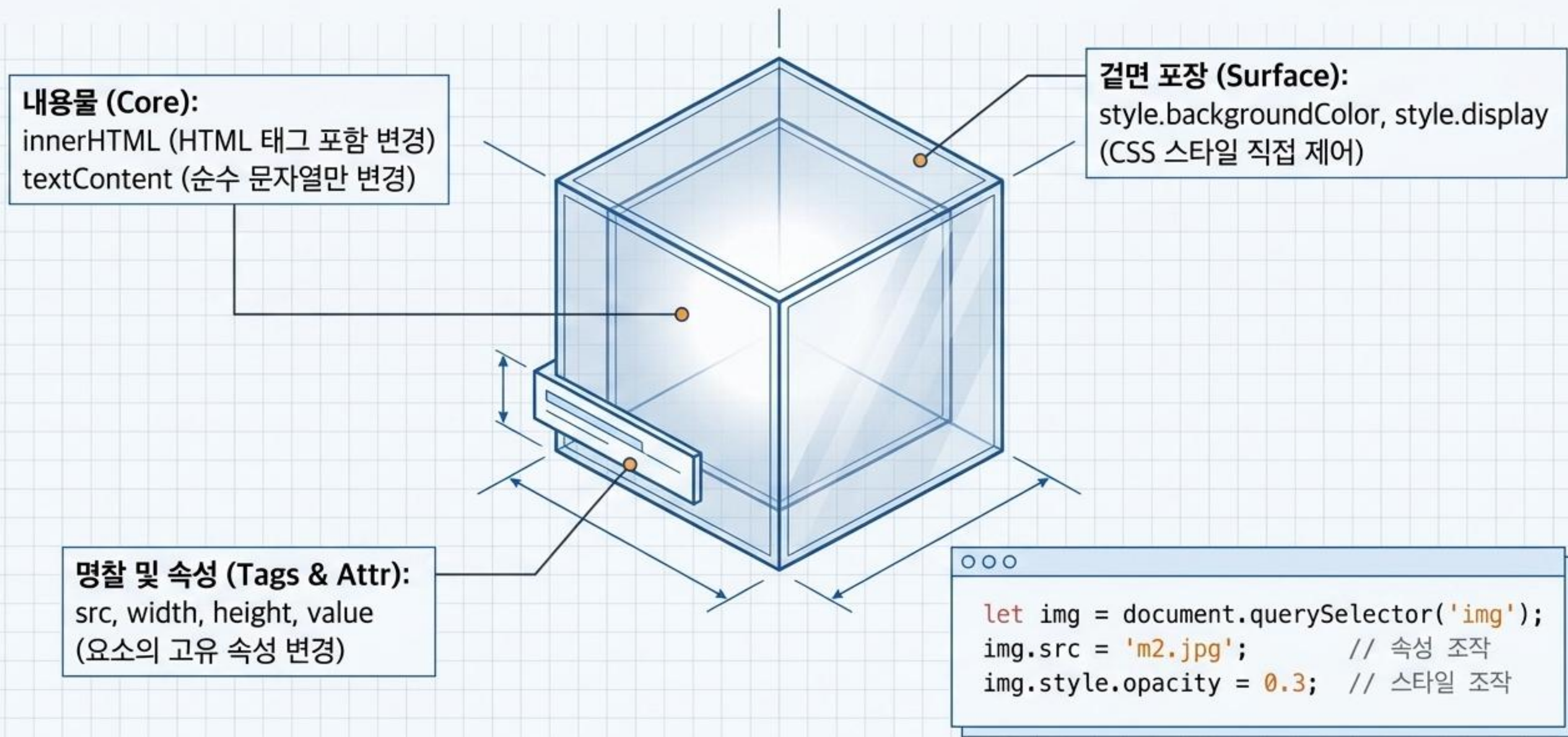
[비교 매트릭스] DOM 선택자 진화도

메서드	대상	반환 타입	비고
<code>getElementById('id')</code>	ID	단일 객체 (Element)	레거시. 빠름.
<code>getElementsByClassName('class')</code>	Class	배열 형태	레거시.
<code>querySelector('CSS선택자')</code>	CSS 문법	첫 번째 단일 객체	모던 웹 권장. 자유로운 탐색.
<code>querySelectorAll('CSS선택자')</code>	CSS 문법	NodeList (배열처럼 순회 가능)	for(let i of h2s)로 제어.

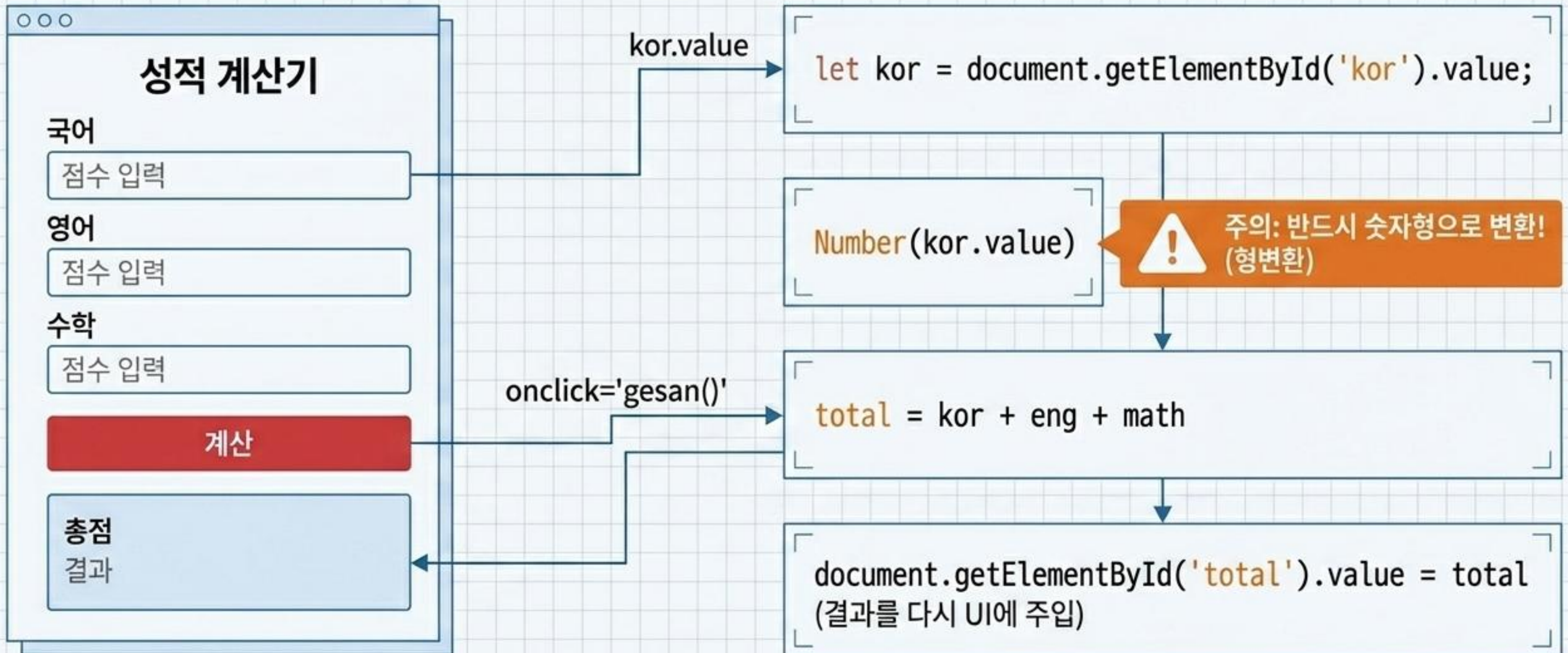


주의: 다중 요소를 반환하는 경우, 배열처럼 루프(for문)를 돌려야 각각의 요소를 제어할 수 있습니다.

Step 2: 문서 객체 모델(DOM) 해부학



UI-to-Code 맵핑: 성적 계산기 작동 원리

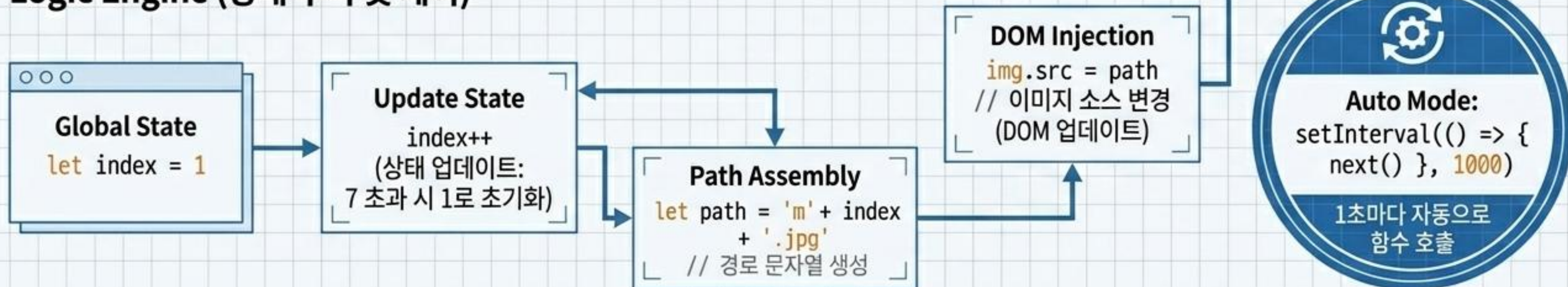


동적 속성 제어: 이미지 갤러리 상태 추적

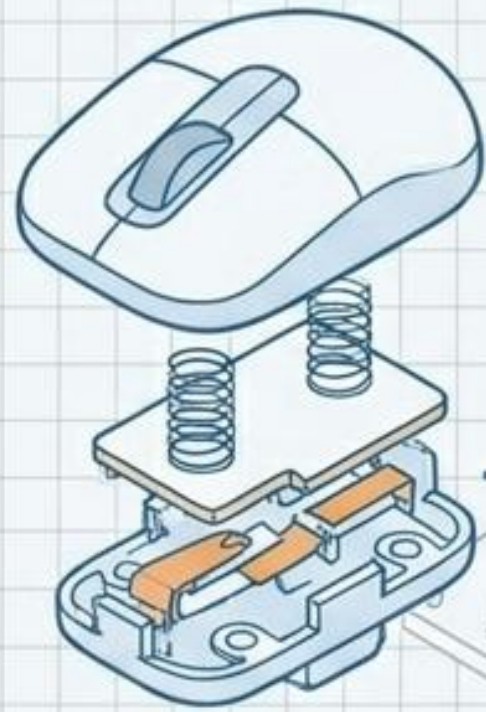
UI (User Interface)



Logic Engine (상태 추적 및 제어)

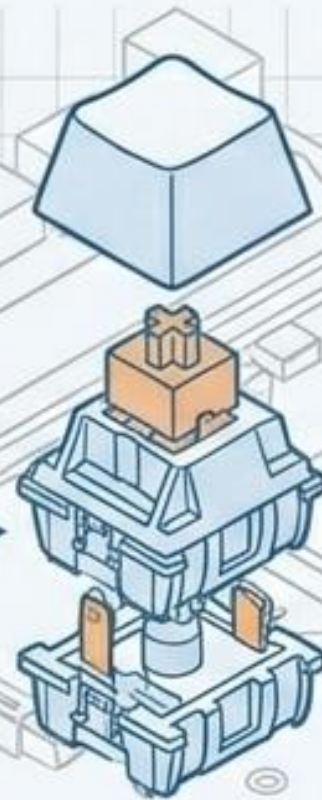


Step 3: 이벤트 (사용자 반응 센서 장착)



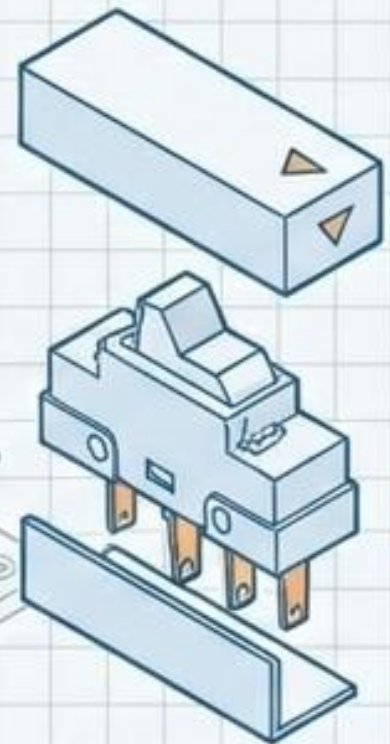
마우스 (Mouse)

onclick (클릭)
onmouseover / onmouseout
(진입/이탈 감지)



키보드 (Keyboard)

onkeydown / onkeyup
(타이핑 감지)



폼 (Form)

onchange
(수량 선택 등 상태 변경 감지)

“이벤트는 웹페이지가 사용자의 행동에 귀를 기울이게 만드는 신경망(Neural Network)입니다.”

[비교 매트릭스] 이벤트 바인딩 3대장

인라인 (Inline)

```
<button onclick='btnClick()'>
```

HTML 안에 JS가 섞여 있어
유지보수 저하.

고전 방식 (Classic)

```
btn.onclick = function()
```

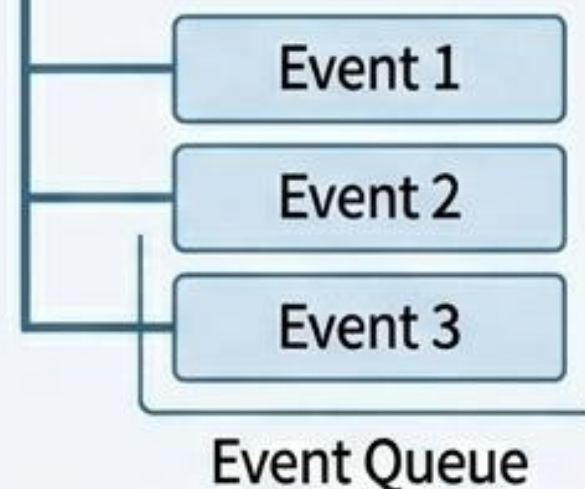


JS 파일 분리 가능.
하지만 치명적 단점 존재.

동일 이벤트 할당 시 마지막 함수가
이전 함수를 덮어쓰 (Override).

이벤트 리스너 (Listener) [모던 웹 권장]

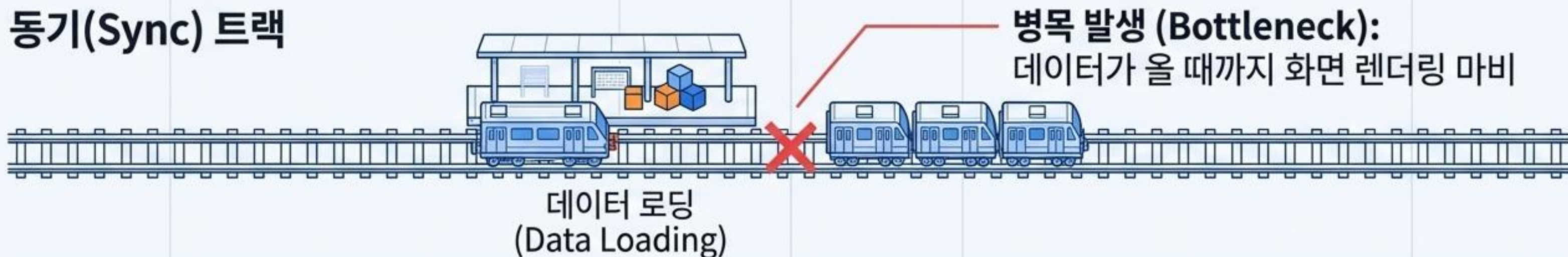
```
btn.addEventListener('click', fn)
```



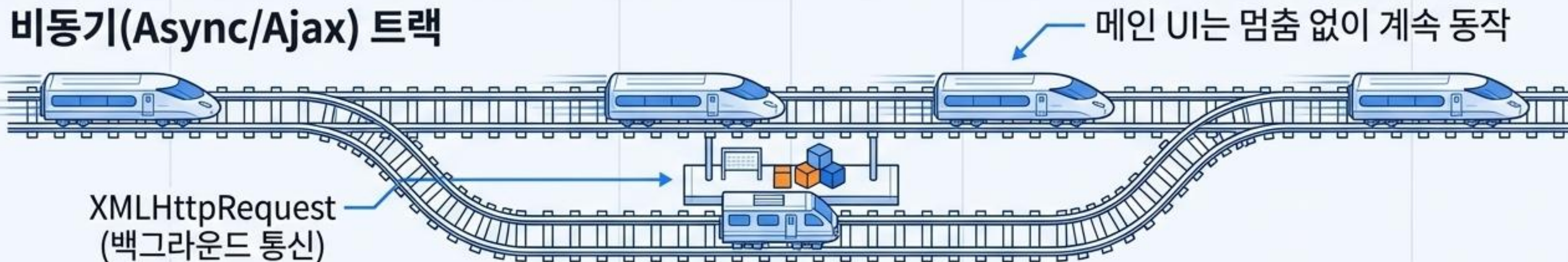
여러 개의 이벤트를 차곡차곡
쌓아서(Stacking) 모두 실행 가능.
jQuery의 .on()과 유사.

Step 4: 비동기(Async)의 본질 - 페이지 멈춤 없는 데이터 통신

동기(Sync) 트랙



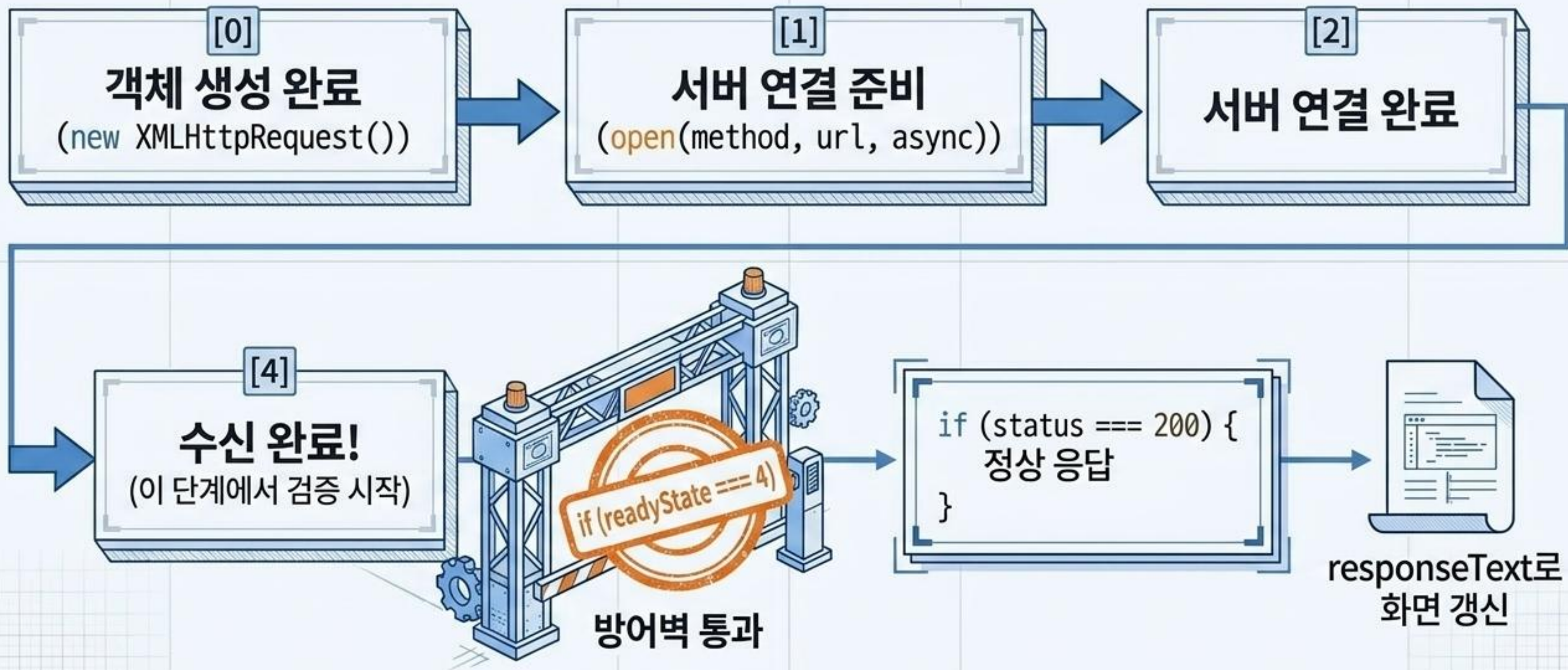
비동기(Async/Ajax) 트랙



Ajax (Asynchronous JavaScript and XML)

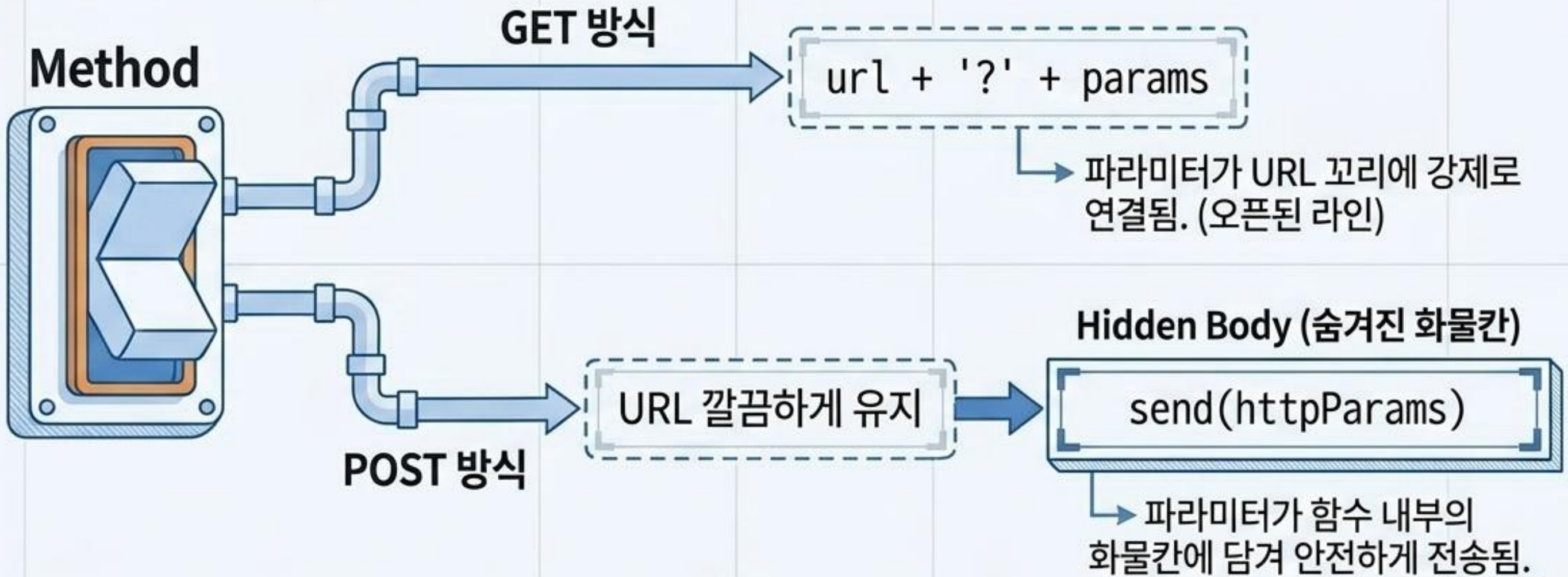
화면 전체를 새로고침하지 않고, 백그라운드에서 필요한 데이터만 서버와 몰래 교환하는 기술.

XMLHttpRequest 생명주기 (Life Cycle) 파이프라인



데이터 송신 설정: GET vs POST 라우팅

Transmission Setup Panel



주의: POST 전송 시 한글 깨짐 방지를 위한 필수 필터

`httpRequest.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded')`

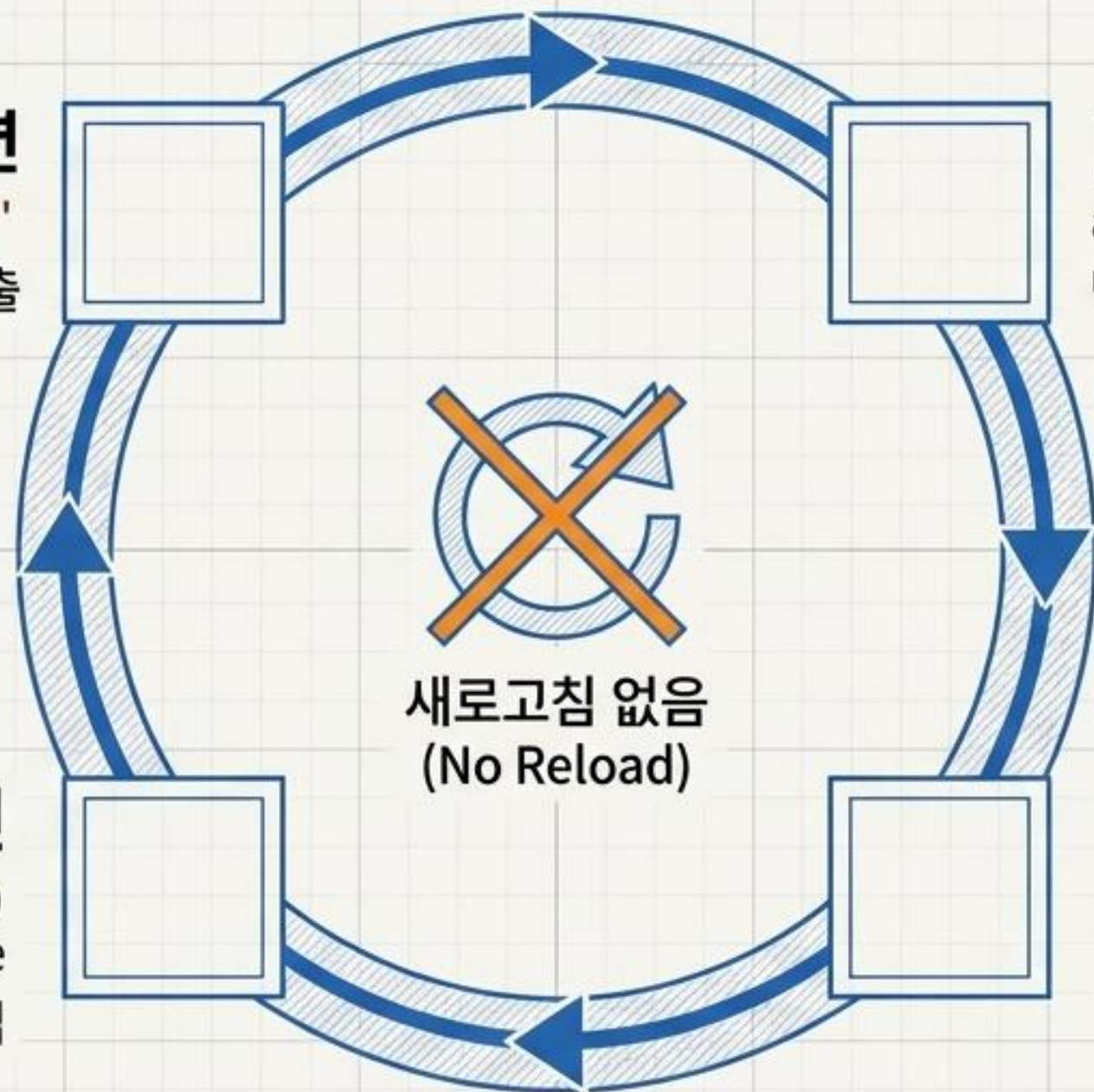
통합 시스템: 인터랙티브 웹의 순환 메커니즘

[Event] 사용자 액션
`onmouseover='detailData(empno)'`
마우스를 올리면 사번 추출

[Ajax] 데이터 요청
`axios.get('./emp/detail.do', {empno})`
백그라운드 통신

[DOM] 화면 갱신
`document.getElementById('name').innerHTML = detail.ename`
특정 태그에만 데이터 주입

[Object] 데이터 수신
`detail = response.data`
서버에서 상세 정보(JSON) 도착



이 4단계가 물 흐르듯 순환하며 화면 깜빡임 없는 모던 웹 애플리케이션을 완성합니다.

[언어 번역기] Vanilla JS vs jQuery 로제타 스톤

	순수 자바스크립트 (Vanilla JS)	제이쿼리 (jQuery)
[선택]	<code>document.querySelector('#btn')</code>	<code>\$('#btn')</code>
[내용 쓰기]	<code>태그.innerHTML = html</code>	<code>\$().html(html)</code>
[내용 읽기]	<code>태그.textContent</code>	<code>\$().text()</code>
[속성 제어]	<code>태그.setAttribute('src', '')</code>	<code>\$().attr('src', '')</code>
[이벤트]	<code>addEventListener('click', fn)</code>	<code>.on('click', fn)</code>

제이쿼리는 내부적으로 자바스크립트 객체 모델을 단순화한 래퍼(Wrapper) 라이브러리입니다.
바닐라 JS의 원리를 알면 어떤 프레임워크도 다룰 수 있습니다.

웹 컨트롤 센터 마스터 블루프린트 (Cheat Sheet)

탐색기 (DOM Selection)

- id: `#id명`
- class: `.class명`
- 태그: `태그명`
- 다중 요소(`querySelectorAll`)는 루프 순회 필수

조작부 (DOM Manipulation)

- 내용: `innerHTML`(태그 포함), `textContent`(텍스트만)
- 스타일: `태그.style.color`
- 속성: `태그.value`, `태그.src`

센서부 (Event Binding)

- 마우스: `click`, `mouseover`, `mouseout`
- 키보드: `keydown`, `keyup`
- 리스너 권장: `addEventListener` (덮어쓰기 방지)

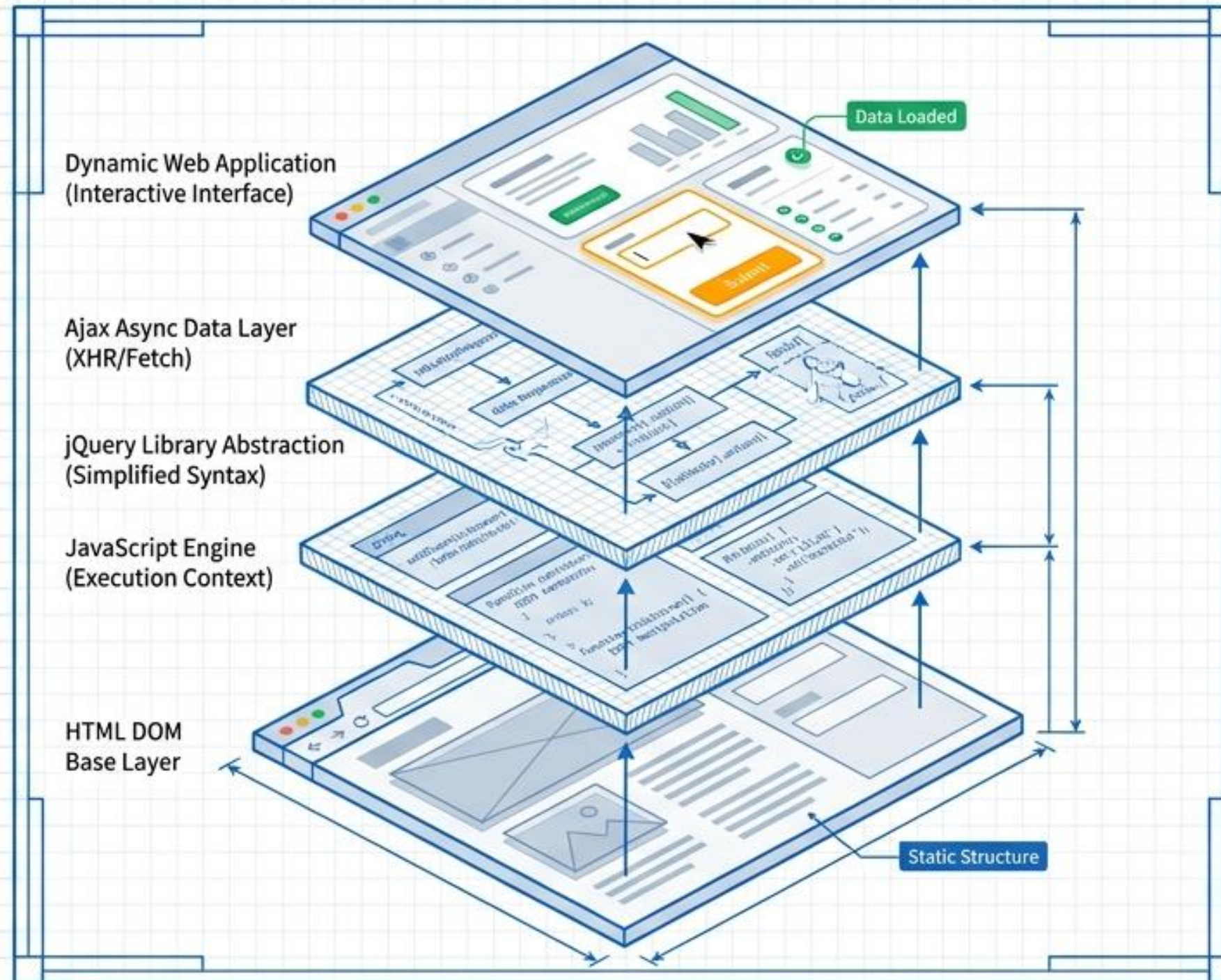
통신망 (Ajax & Async)

- 역할: 화면 새로고침 없는 무중단 데이터 통신
- 단계: `open()` -> `send()` -> `readyState===4`
-> `status===200`

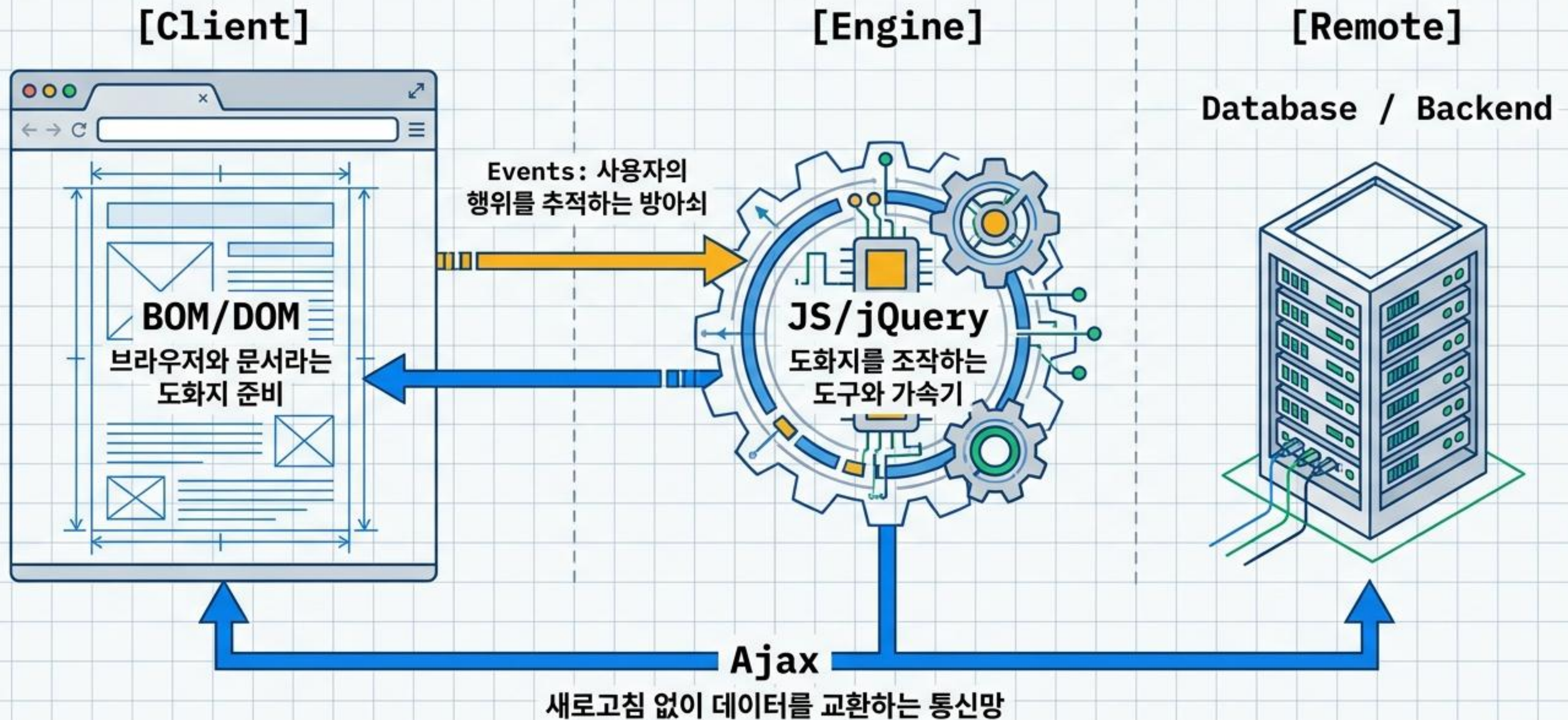
SYSTEM ONLINE. 완벽한 동적 웹페이지를 조작할 준비가 완료되었습니다.

웹 인터랙션 블루프린트

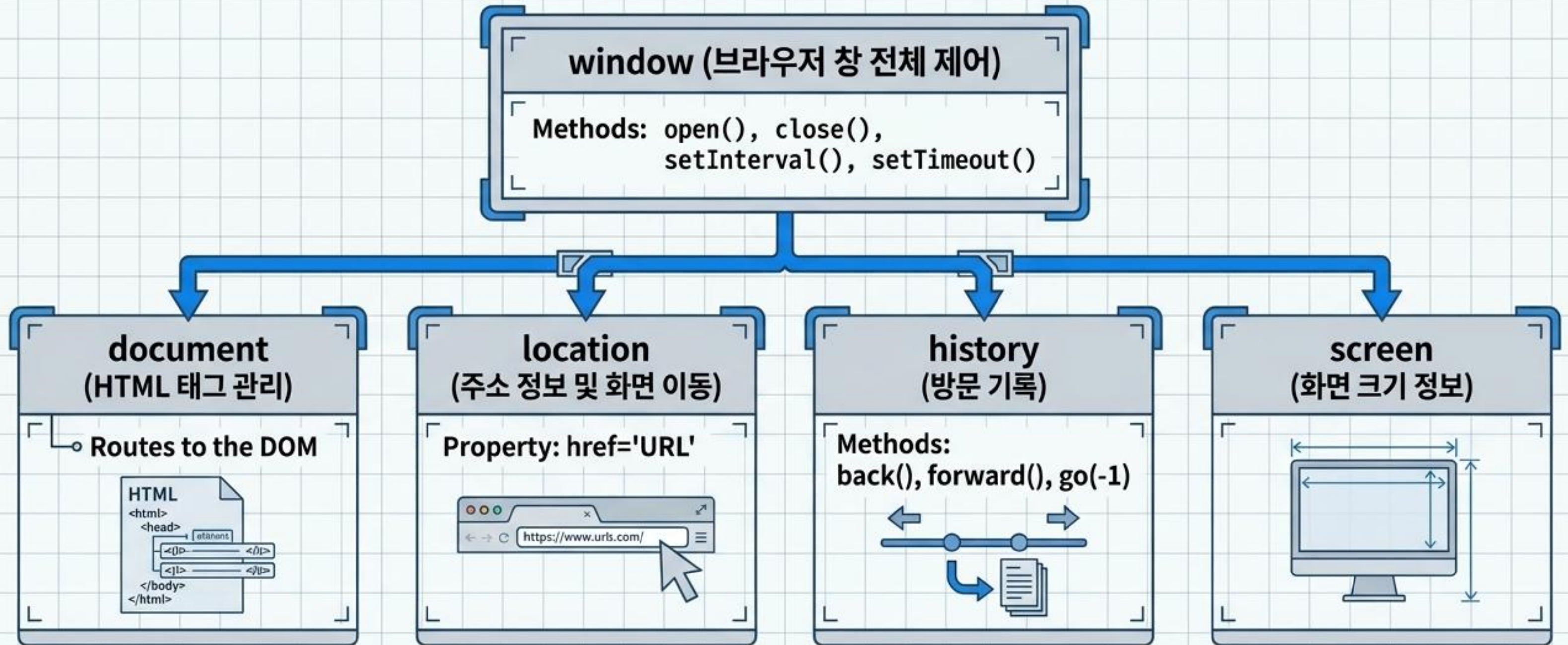
정적인 HTML 도화지를 동적인 애플리케이션으로 진화시키는 JavaScript, jQuery, Ajax의 작동 원리



웹 생태계의 4가지 핵심 기둥



BOM 해부도: 브라우저 객체 모델 (Browser Object Model)



브라우저의 모든 요소는 자바스크립트가 제어할 수 있는 '객체(Object)' 형태로 존재합니다.

DOM 타겟팅 로직: 원하는 요소를 정확히 쏘아 올리기

HTML Source

```
<body>
  <div>
    <h1 id='title' class='text'>
    <h1 class='text'></h1>
  </div>
</body>
```

ID (단일 요소)
Code: `document.getElementById('title')` / `document.querySelector('#title')`

Class (다중 요소)
Code: `document.getElementsByClassName('text')` / `document.querySelectorAll('.text')`

Tag (모든 태그)
Code: `document.getElementsByTagName('div')` / `document.querySelectorAll('div')`

최신 실무에서는 CSS 선택자를 그대로 사용할 수 있는 `querySelector`와 `querySelectorAll`로 타겟팅을 통일합니다.

DOM 조작의 3대 요소 (The 3 Pillars of Manipulation)

글자 조작 (Text/Content)

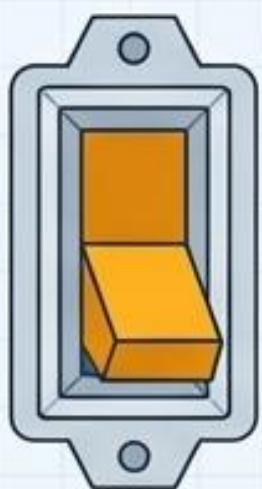
태그 사이의 텍스트나 내부 HTML 변경

Methods:

`textContent`, `innerHTML`

Example:

```
h1.innerHTML = 'Hello JS!!'
```



[Hello JS!!]

스타일 조작 (Style)

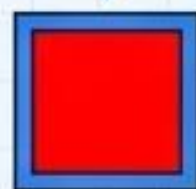
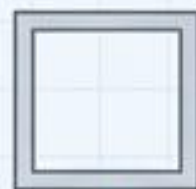
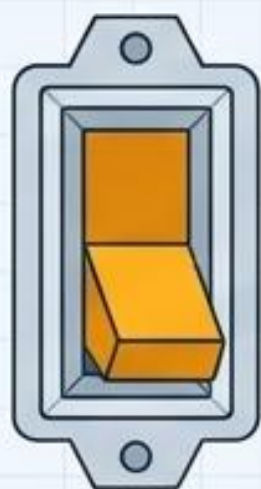
CSS 속성을 등적으로 변경 (카멜 표기법 사용)

Methods:

`element.style.backgroundColor`

Example:

```
h1.style.background = 'red'
```



속성 조작 (Attributes)

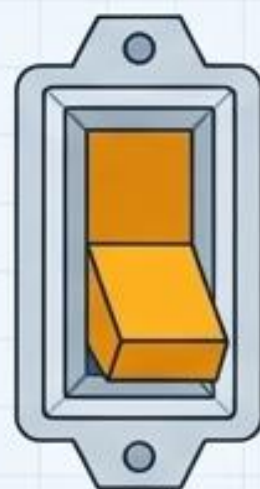
HTML 태그의 고유 속성(src, width, href) 제어

Methods:

`setAttribute()`, `getAttribute()`

Example:

```
img.src = '../image/m2.jpg'
```



자바스크립트 기본 튜벨트: String & Math 객체

String 객체 (문자열 가공)



Extract (자르기):

substring(시작, 끝)
substr(시작, 갯수)
slice()

Search (찾기):

indexOf()
lastIndexOf()

Refine (정제):

trim() (좌우 공백 제거)
replace() (문자 변환)

Transform (변환):

split() (특정 구분자로
나누어 배열 반환)

Math 객체 (수치 연산)



Note: 정적(static) 객체로
new Math() 사용 불가.

Rounding:

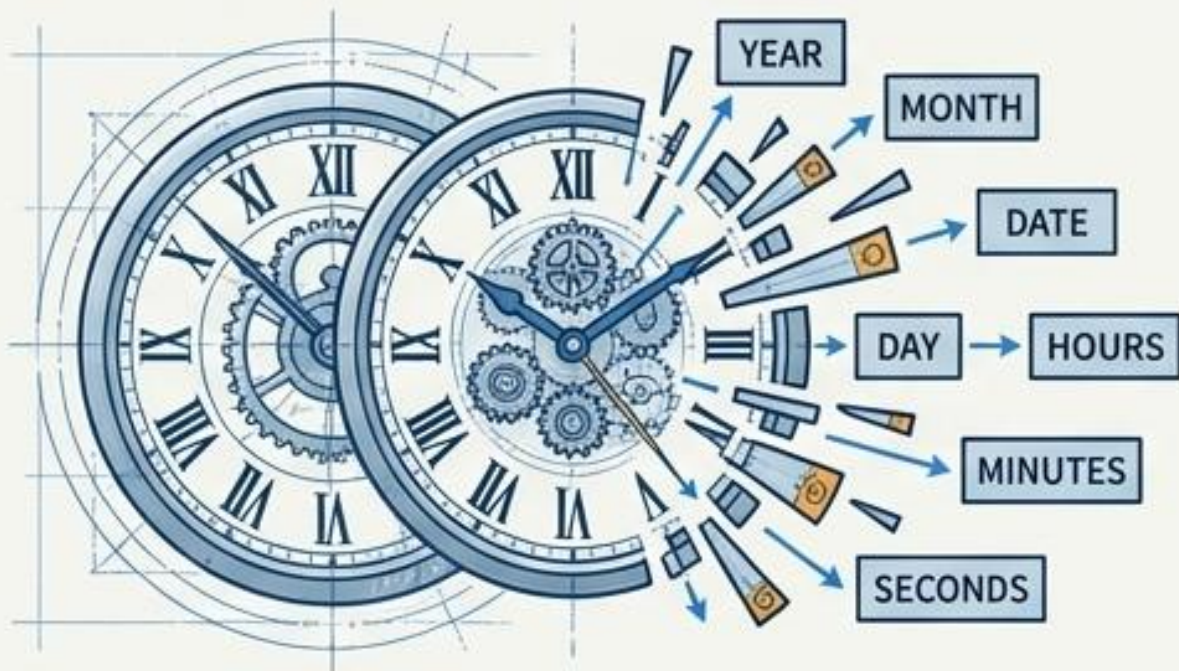
Math.round() (반올림)
Math.ceil() (올림)
Math.floor() (내림)

Random:

Math.random()
(0~1 미만 난수 생성)

데이터 캡슐화: Date 객체와 JSON 메커니즘

Date 객체 (시간의 해체)

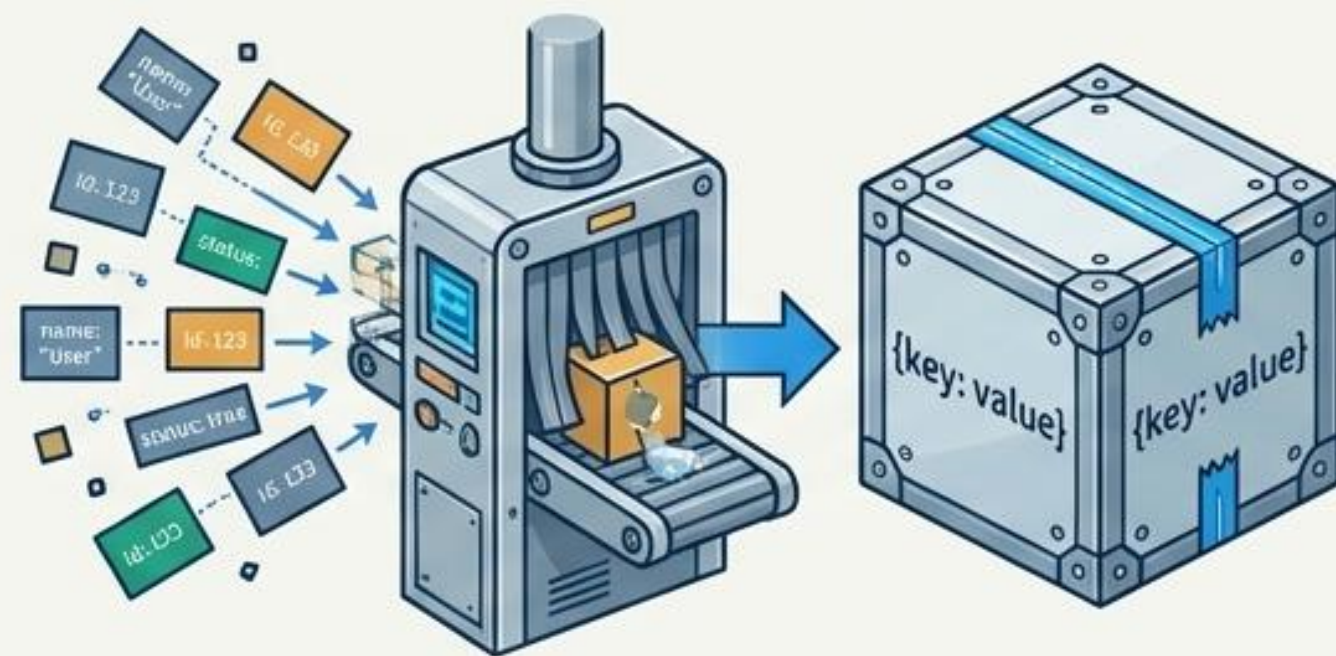


⚙️ **Creation:** `let today = new Date()`

🎯 **Methods:**

- ◆ `getFullYear()`
- ◆ `getMonth()` (Note: 0부터 시작)
- ◆ `getDate()`
- ◆ `getDay()` (0:일 ~ 6:토)

JSON (데이터 교환의 표준)



클라이언트와 서버 간의 데이터 교환을 위한 경량 객체 표기법.

Stringify (포장):  `data payload`
JS 객체 → `JSON.stringify(info)` → 서버 전송용 문자열

Parse (해체):  `data payload` 
서버 응답 문자열 → `JSON.parse(data)` → JS 객체 변환

Array Methods (Quick List): `push()` (끝에 추가), `pop()` (끝 제거), `slice()` (배열 분리)

이벤트 흐름: 사용자의 행위 추적 및 대응

Step 1: Trigger (이벤트 발생)



- Types:
onclick, onchange,
onkeyup, onmouseover.

Step 2: Listener (감지기 부착)



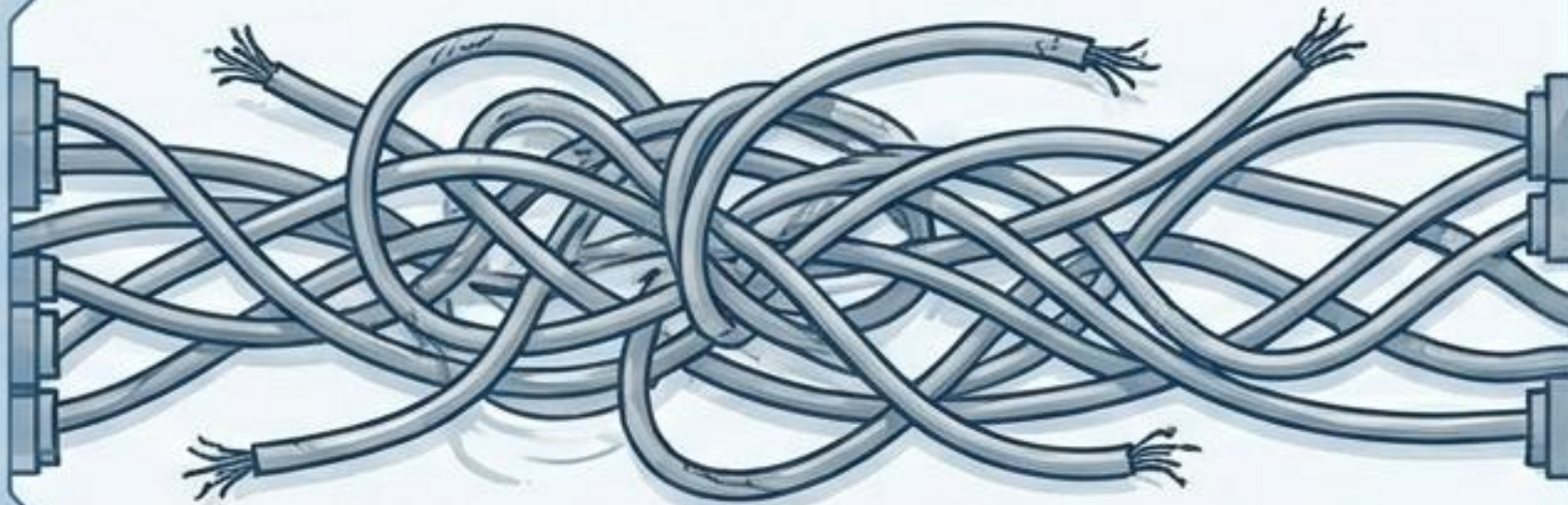
- Classic:
`btn.onclick = function() { ... }`
- Modern:
`btn.addEventListener('click', ...)`

Step 3: Execution (함수 호출)



- Declarative:
`function name() { ... }`
- Anonymous/Callback:
`let func = function() { ... }`
- Arrow (ES6):
`let func = () => { ... }`

패러다임 시프트: 왜 jQuery인가?



The Pain (Vanilla JS)

DOM 요소를 조작하려면 항상 반복문(for loop)이 필요합니다. 코드가 길어지고 가독성이 떨어집니다.

```
let h1 = document.querySelectorAll('h1');  
for(let i of h1) { i.style.background = 'red'; }
```

The Solution (jQuery)



Write Less, Do More. 내부적으로 반복문을 처리하여 단 한 줄로 다중 요소 제어가 가능합니다.

```
$('#h1').css('background', 'red');
```

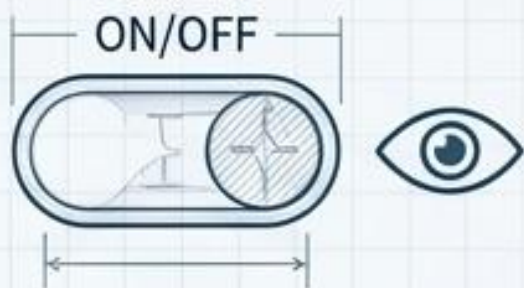
Core Concept: \$ 기호 하나로 querySelector를 대체하고, 강력한 체이닝(Chaining)을 지원합니다.

바닐라 JS vs jQuery: 코드 다이어트 매트릭스

50 mm	76 mm	Vanilla JS	jQuery
		요소를 여러 개 선택할 때 <code>document.querySelectorAll('.class')</code>	요소를 여러 개 선택할 때 <code>\$('.class')</code>
		텍스트 변경 <code>element.textContent = '값' / element.innerHTML = '값'</code>	텍스트 변경 <code>\$(selector).text('값') / \$(selector).html('값')</code>
		입력값(Value) 읽기/쓰기 <code>input.value / input.value = 'admin'</code>	입력값(Value) 읽기/쓰기 <code>\$('#input').val() / \$('#input').val('admin')</code>
		이벤트 바인딩 <code>btn.addEventListener('click', () => { ... })</code>	이벤트 바인딩 <code>\$('#btn').on('click', function() { ... }) 또는 \$('#btn').click(...)</code>
	200 mm		

jQuery 핵심 UI 제어 도구 (Quick Reference)

가시성 제어 (Visibility)



show() / hide():
CSS의 display: '' 와
display: 'none'을 자동 처리.

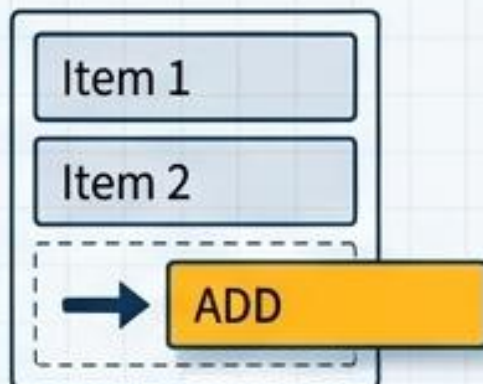
속성 제어 (Attributes & Properties)



.attr('src', 'new.jpg'):
이미지 소스, 링크 등 HTML 속성 변경.

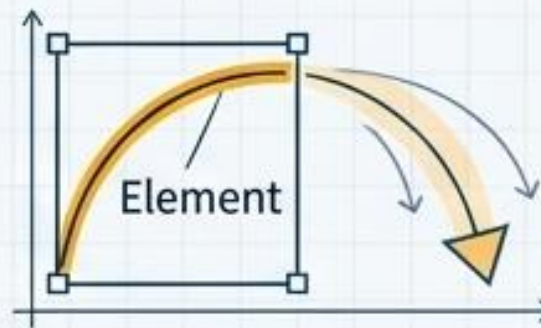
.prop('checked', true):
체크박스 전체 선택/해제 등 상태 제어.

동적 추가 (DOM Insertion)



.append('<tr>...</tr>'):
기존 콘텐츠의 맨 끝에 새로운
HTML 태그를 지속적으로
누적 추가 (장바구니 구현).

애니메이션 및 마우스 추적



**.hover(function(){...},
function(){...}):** mouseenter와
mouseleave를 한 번에 처리.

.animate({'top': pos + 'px'}, 1000):
퀵메뉴, 부드러운 스크롤 애니메이션.

동기(Sync) vs 비동기(Async)의 패러다임 차이

동기식 (기존 방식)

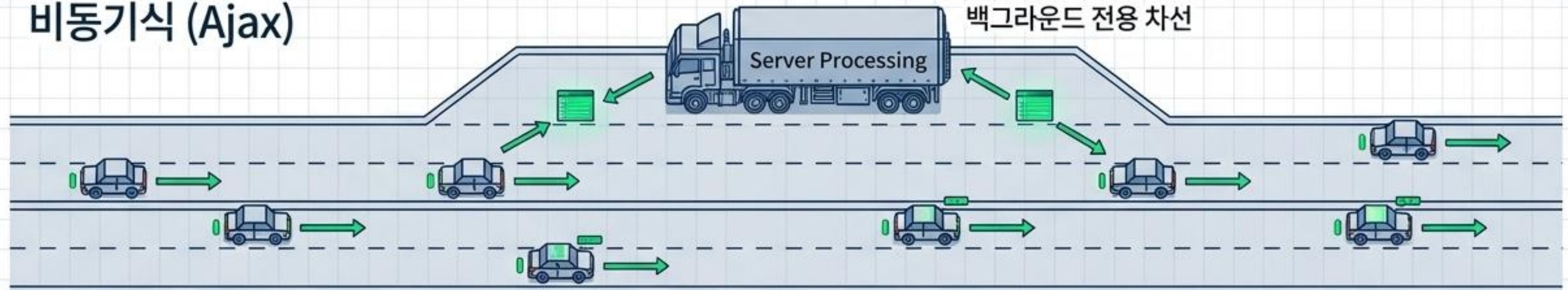


새로고침
(Reload)



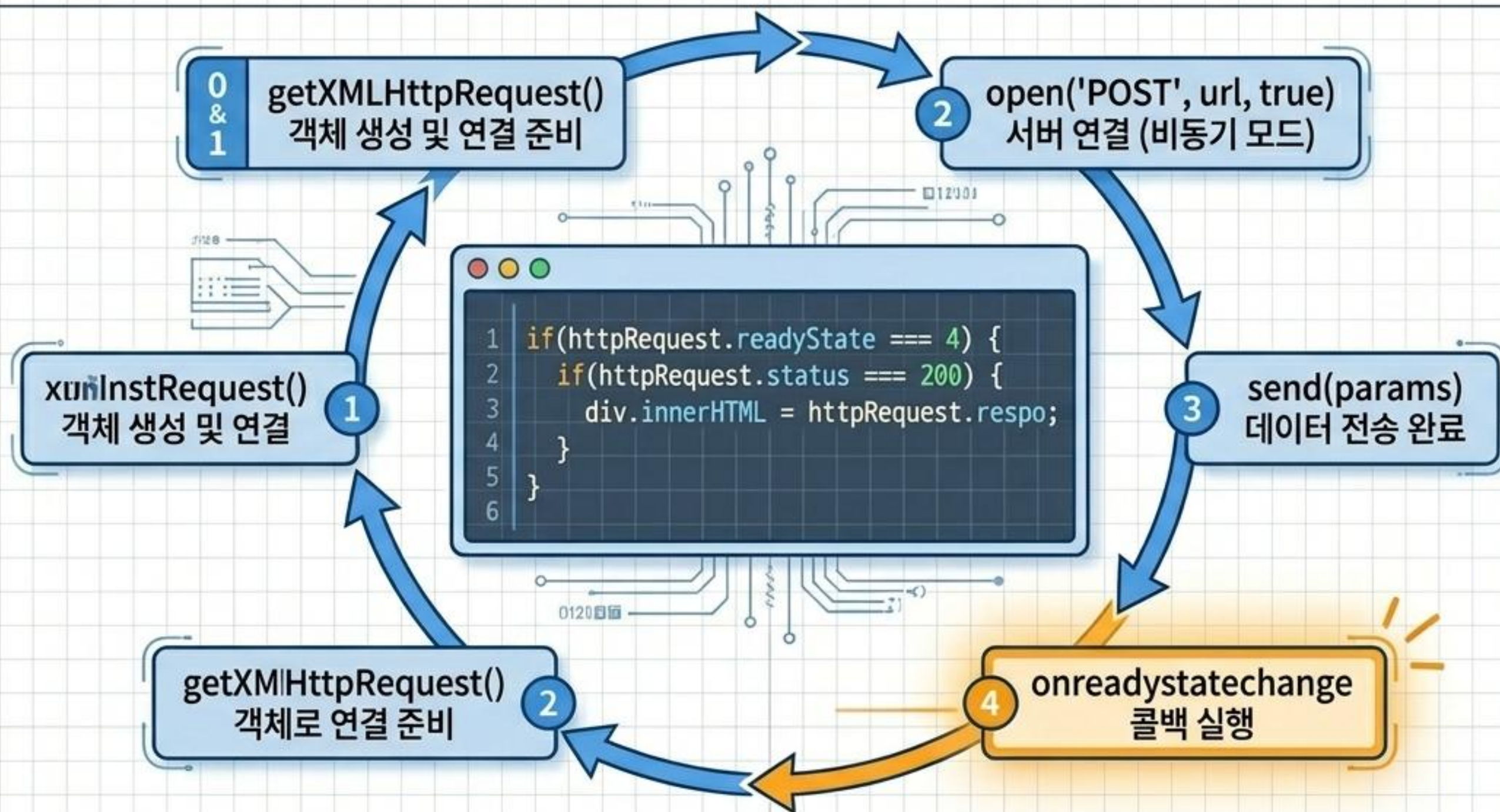
서버에 데이터를 요청하면 응답이 올 때까지 브라우저 화면이 하얗게 멈추고 새로고침이 발생합니다.

비동기식 (Ajax)

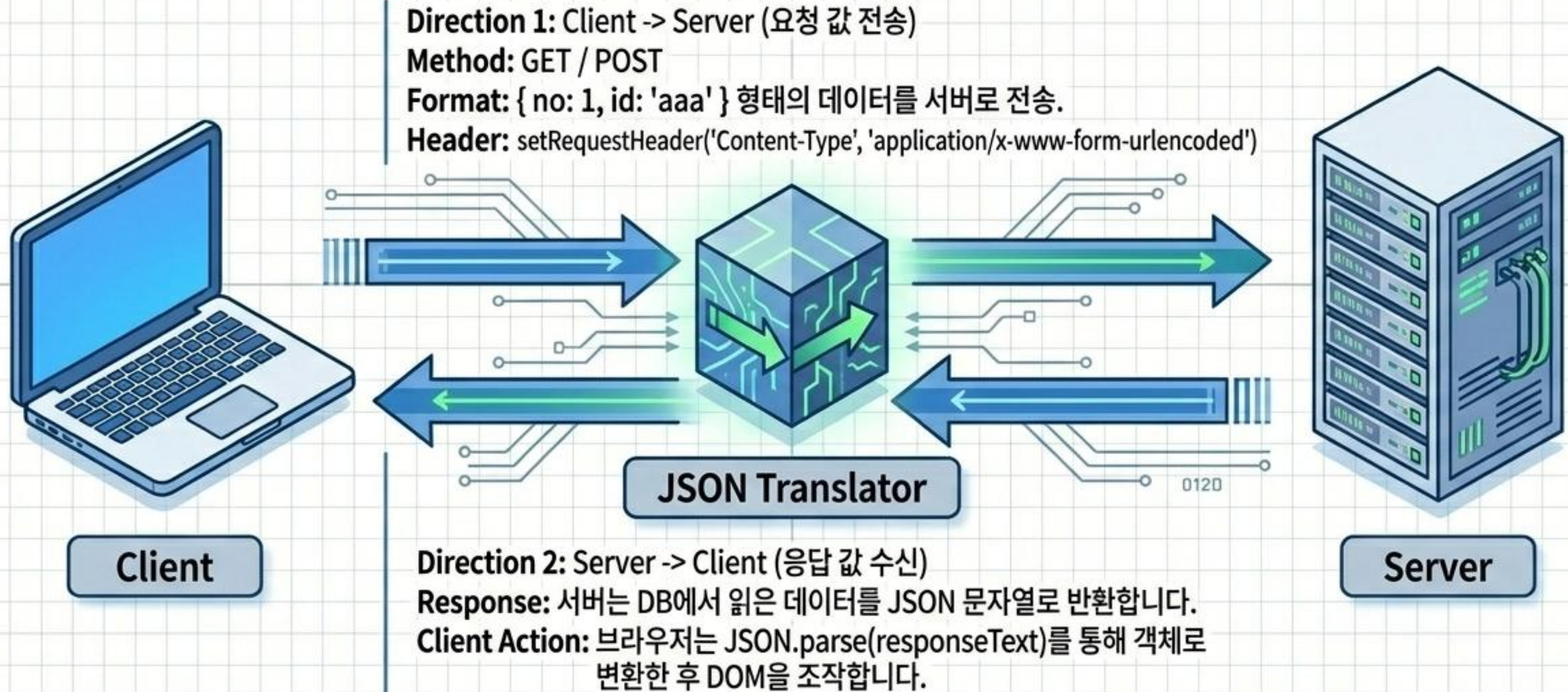


백그라운드 전용 차선을 통해 서버와 데이터를 교환합니다. 화면의 깜빡임 없이 페이지의 부분 업데이트가 가능합니다.

Ajax 작동 아키텍처: XMLHttpRequest 라이프사이클



클라이언트와 서버의 언어: JSON 데이터 교환

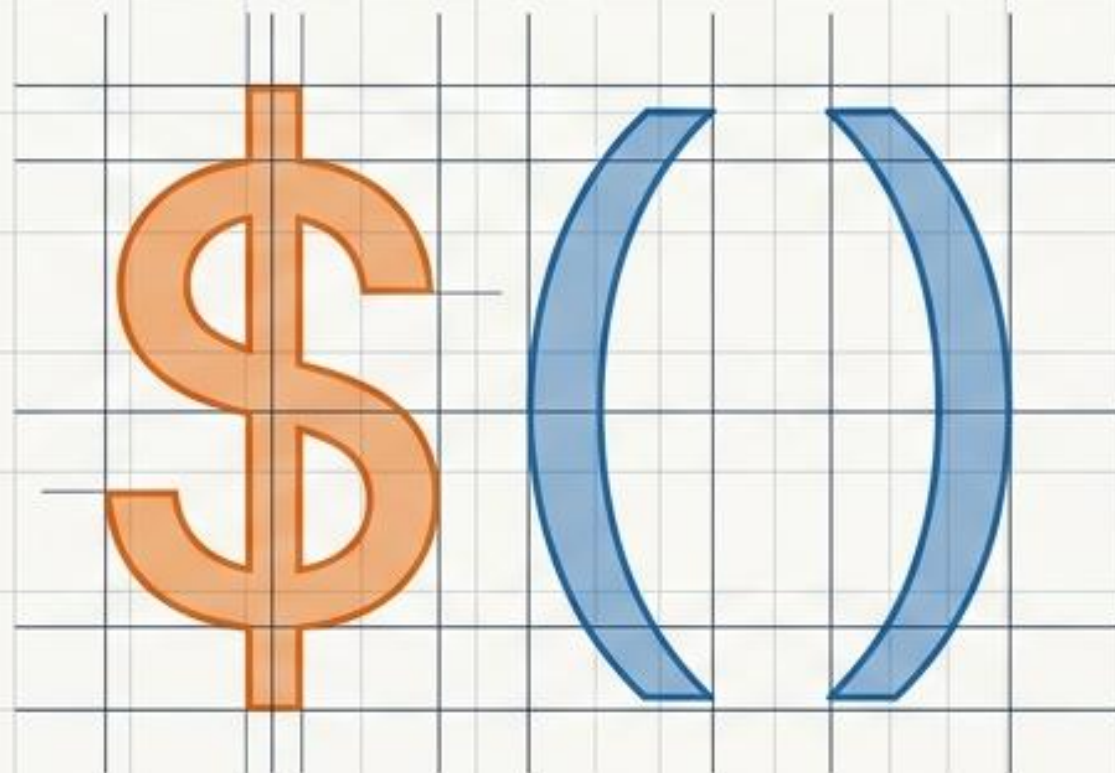


풀스택 인터랙션 시퀀스: From Click to Server

Master Sequence Map



이 완벽한 사이클이 현대적인 단일 페이지 애플리케이션(SPA)과 동적 웹 경험을 가능하게 하는 핵심 뼈대입니다.



jQuery 완전 정리

웹 개발 역사를 바꾼 철학, Write less, do more.

[웹 개발 입문자]

[레거시 시스템 유지보수]

[퍼블리셔]

v3.x 기반

과거의 고통과 jQuery의 등장

과거 순수 JS

```
document.getElementById("title")  
.style.color = "red";
```

브라우저마다 다른 동작
(크로스 브라우징 문제)

길고 복잡한 DOM 접근

jQuery의 혁명

```
$("#title").css("color", "red");
```

짧고 직관적인 문법

복잡한 DOM 조작과 브라우저 호환성 문제를 한 줄의 직관적인 코드로 해결.

코어 문법 해부학 (The Anatomy)



안전 장치: DOM Ready

HTML이 모두 로드된 후 실행되어 에러 방지

표준: `$(document).ready(function() {});`

축약: `$(function() {});`

jQuery 객체

jQuery 기능을 호출하는
팩토리 함수

`$(selector).action();`

선택자

조작할 HTML 요소를 타겟팅

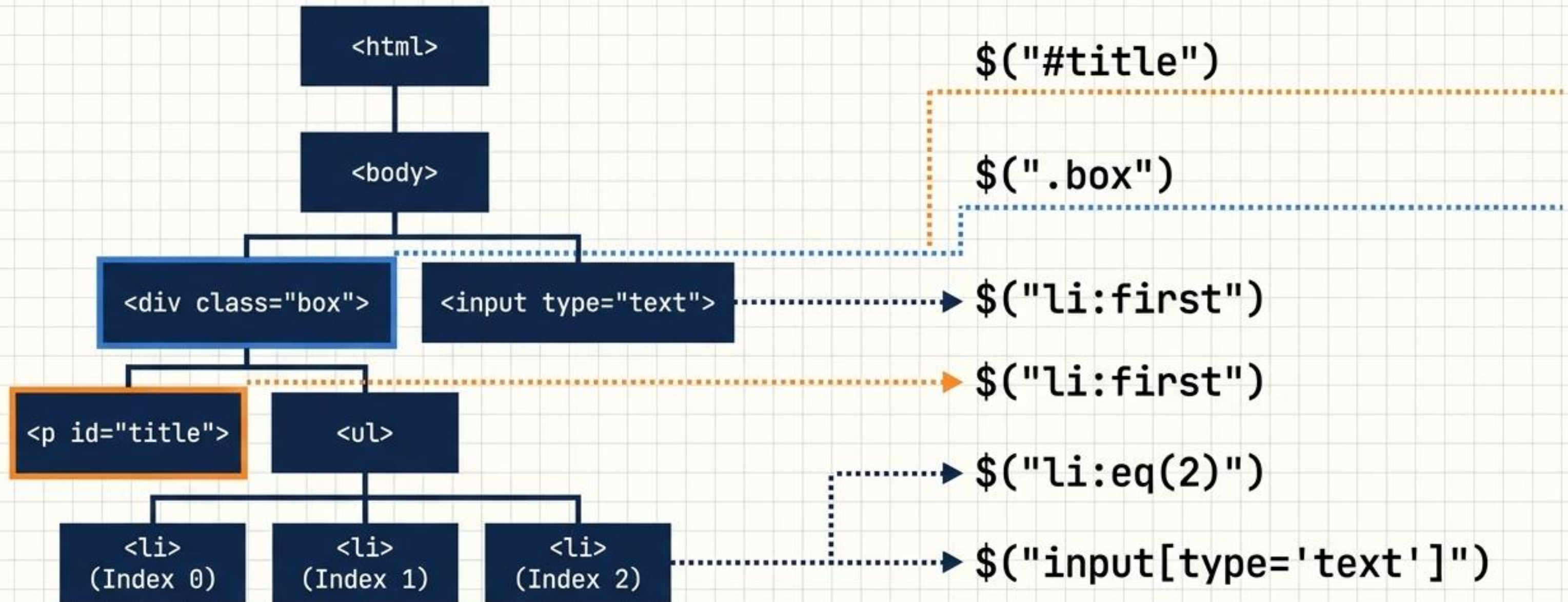
수행 작업

선택된 요소에 수행할 이벤트,
CSS, 효과 등

실무 표준 CDN 방식:

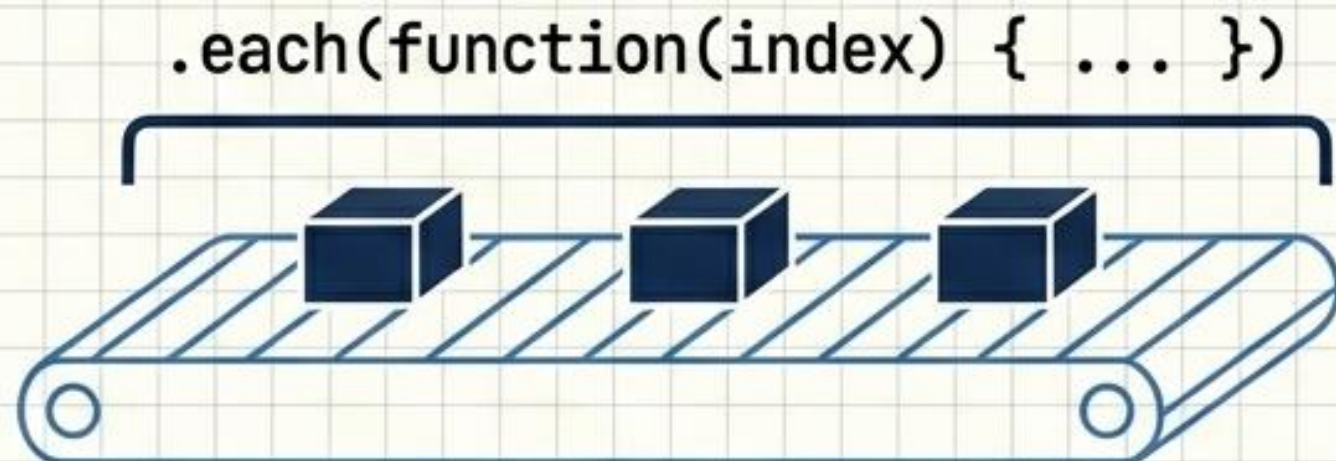
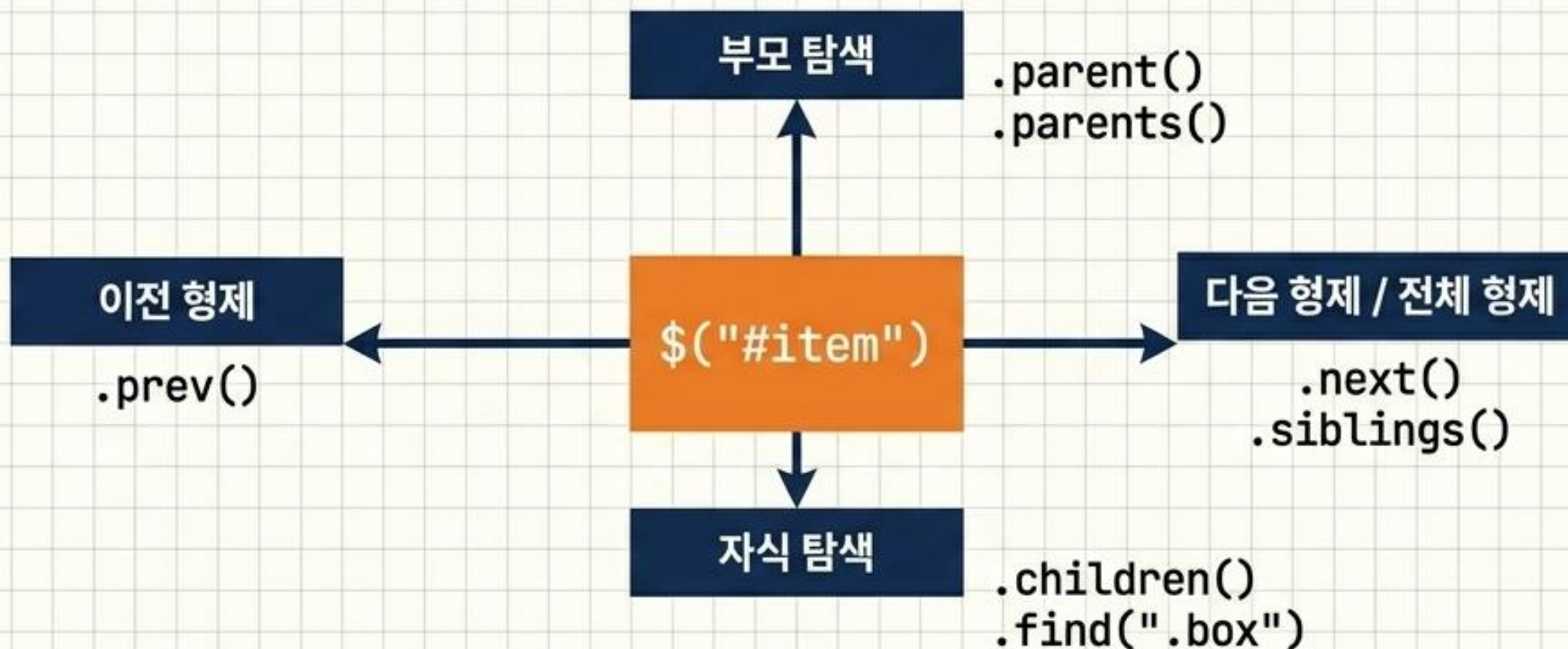
```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```


타겟팅: DOM 트리 내비게이션 (Selectors)



Insight: jQuery의 핵심은 '원하는 요소를 얼마나 빠르고 정확하게 찾아내는가'에 있습니다.

트리 탐색 (Traversing & Iteration)



다중 요소를 선택한 후, 컨베이어 벨트처럼 하나씩 순회하며 반복 작업을 수행합니다.

구조와 내용 조작 (DOM Manipulation)

내용 변경



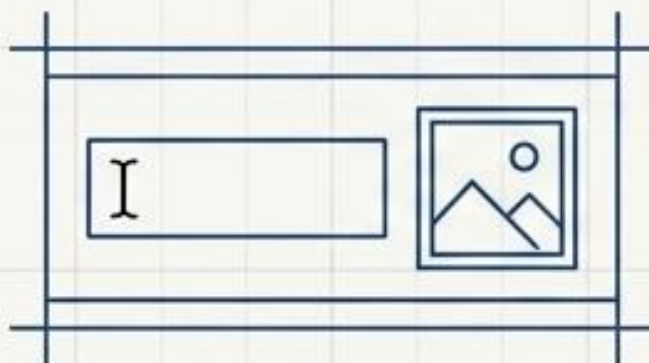
```
.text("안녕하세요");
```

(단순 텍스트)

```
.html("<b>굵게</b>");
```

(HTML 태그 포함)

값 및 속성 제어



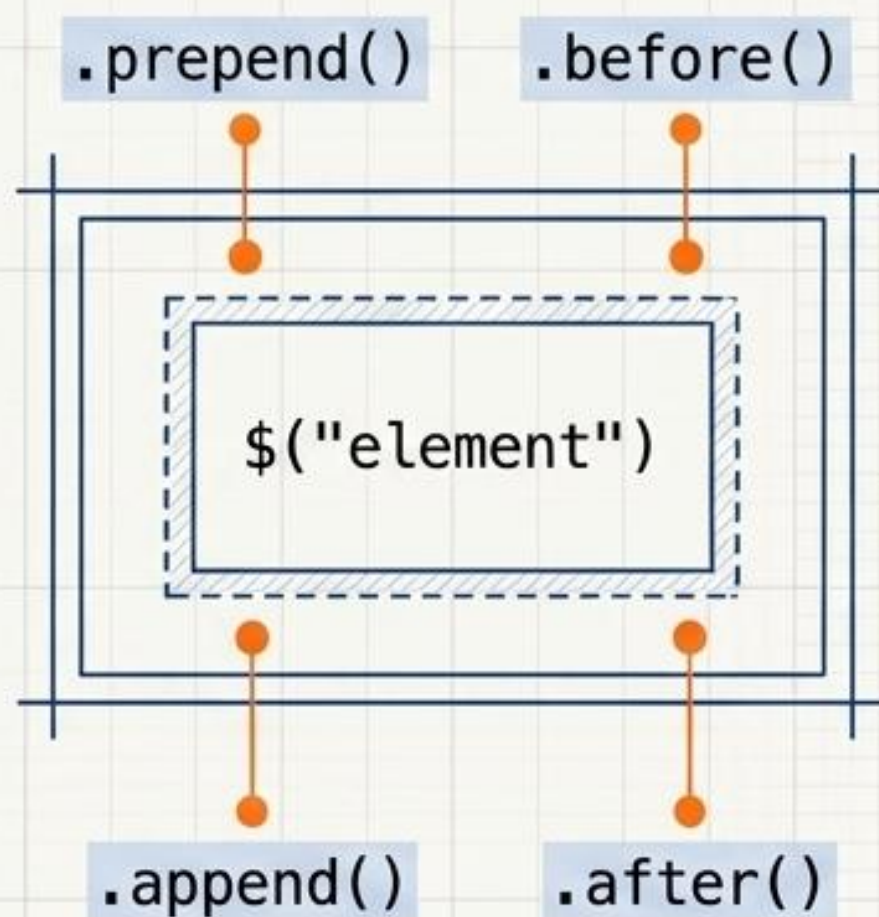
```
.val();
```

(폼 입력값 읽기/쓰기)

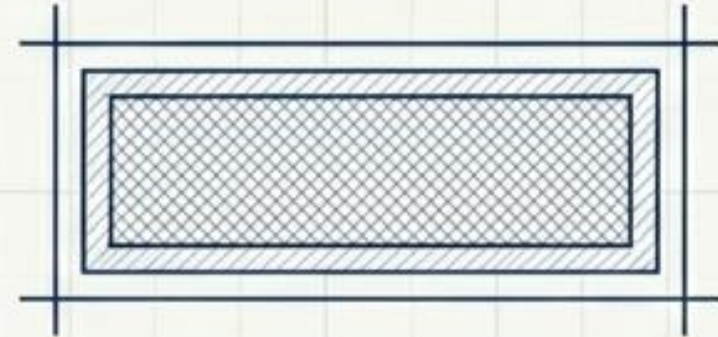
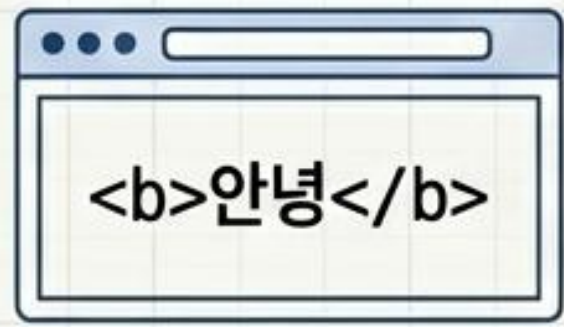
```
.attr("src", "test.jpg");
```

(속성 변경)

요소 삽입 지점

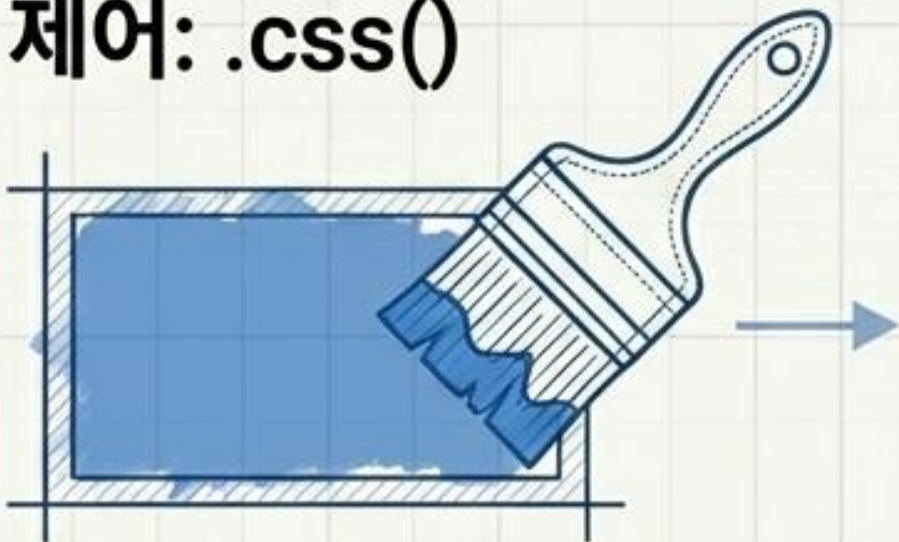


헛갈리기 쉬운 조작 메서드 비교표

Deletion Methods	<code>.remove()</code>  요소 자체를 완전히 삭제 (DOM에서 제거)	<code>.empty()</code>  요소의 형태는 유지한 채 내부 내용물만 삭제
Content Methods	<code>.text("안녕")</code>  HTML 태그를 해석하지 않고 문자열 그대로 출력	<code>.html("안녕")</code>  문자열 내의 HTML 태그를 실제 DOM으로 해석하여 렌더링

스타일과 상태 제어 (CSS & Classes)

직접 제어: .css()



단일: `.css("color", "red");`

다중: `.css({ color: "white", background: "black" });`

상태 제어 (권장 방식)



`.addClass("active")`

클래스 호재 덮
이적한 내용을 읽기 가능



`.removeClass("active")`

클래스 보다를 맞
판나의 상태를 검기



`.toggleClass("active")`

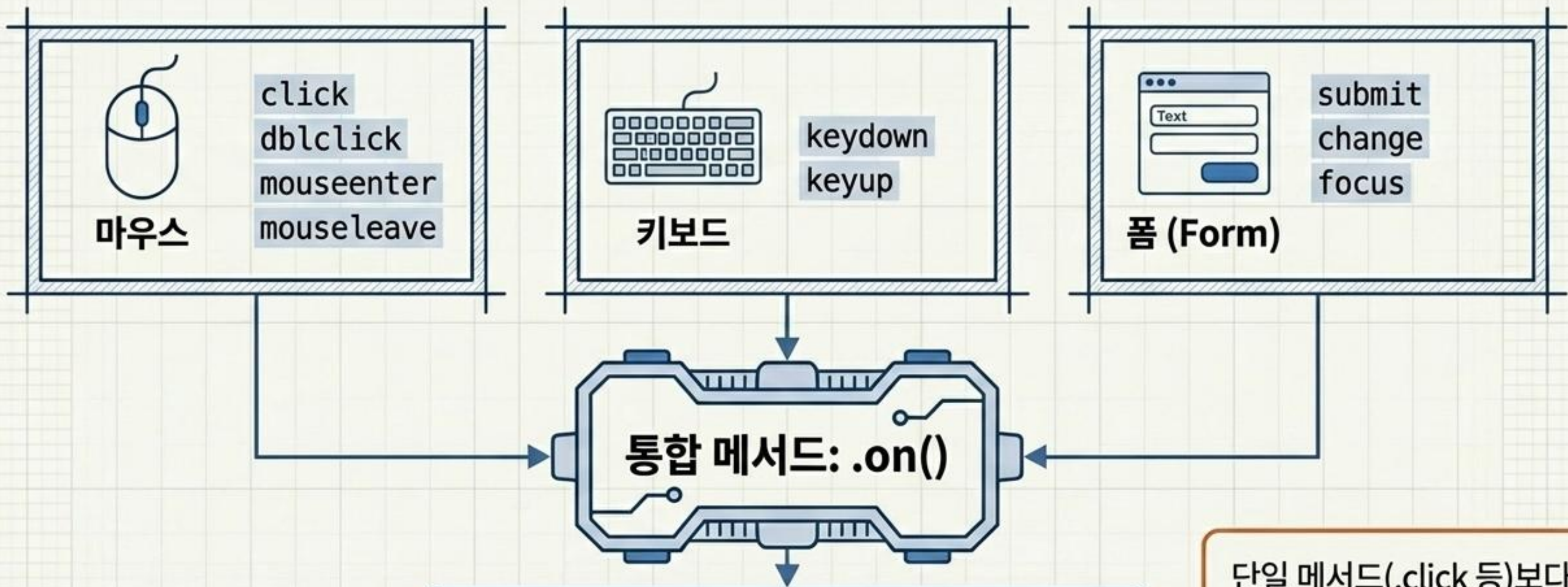
다이를 면장 해보에
그라와 자영을 출리 제기



`.hasClass("active")`

클래스 존재 여부를
True / False로 반환하여 검증

인터랙션의 핵심: 이벤트 (Events)



```
$("#btn").on("click", function() {  
    // 이벤트 발생 시 실행될 로직  
});
```

단일 메서드(.click 등)보다
다중 이벤트 바인딩과
동적 요소 관리에 탁월한
현대적 표준 방식입니다.

실무 필수 패턴: 이벤트 위임 (Event Delegation)

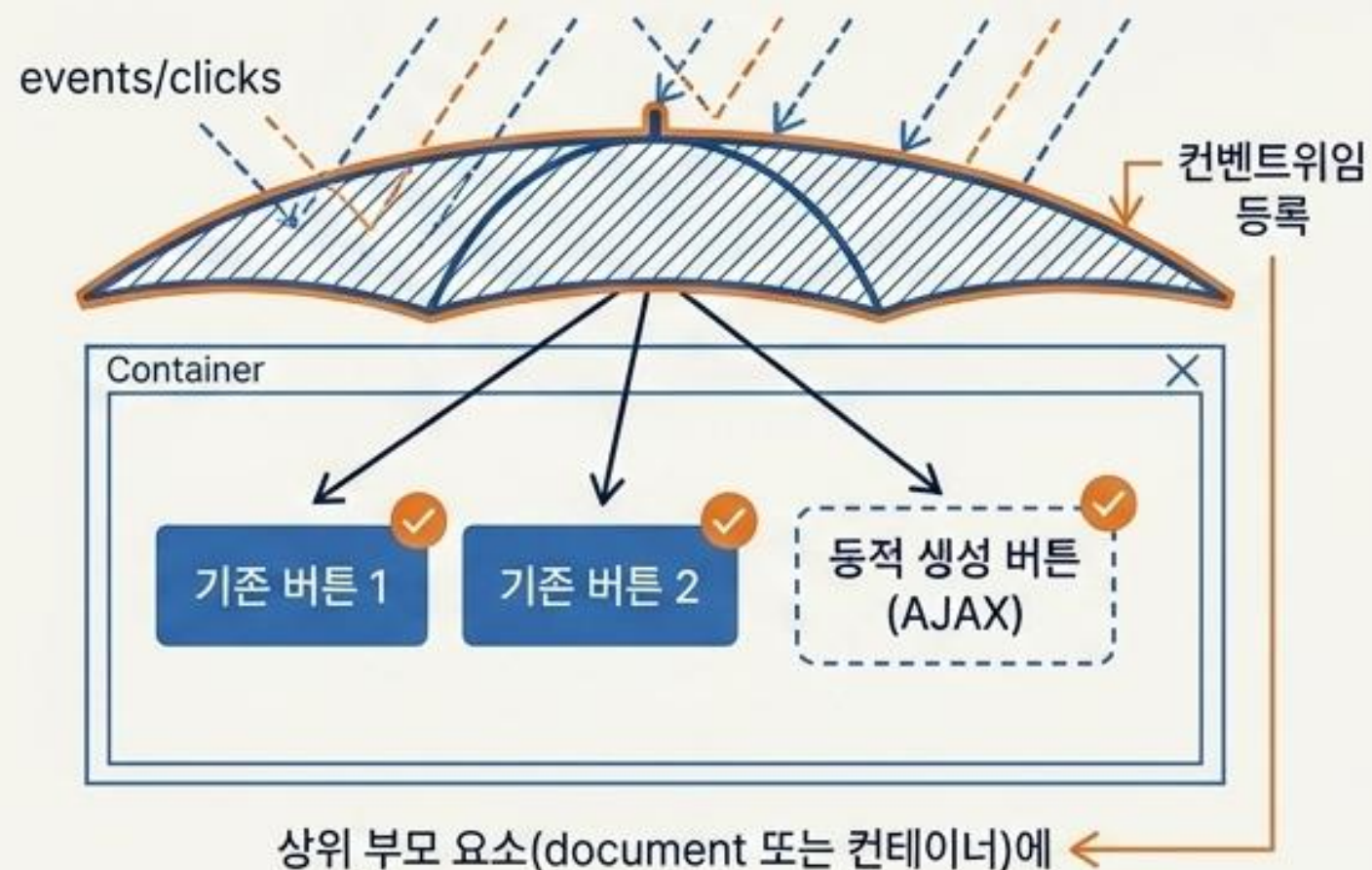
문제 상황 (비효율)



```
$(".btn").click(function(){});
```

동적으로 나중에 생성된 요소는 이벤트를 인식하지 못함.

해결책: 이벤트 위임



상위 부모 요소(document 또는 컨테이너)에

```
$(document).on("click", ".btn", function(){});
```

상위 부모 요소(document 또는 컨테이너)에 이벤트를 등록하여, 나중에 생성된 자식 요소들의 이벤트까지 대신 감지하고 처리함.

| 시각적 효과와 애니메이션 (Effects & Animations)



Display (표시/숨김)

```
hide()  
show()  
toggle()
```



Fade (투명도 조절)

```
fadeIn()  
fadeOut()  
fadeToggle()
```



Slide (높이 조절)

```
slideDown()  
slideUp()
```

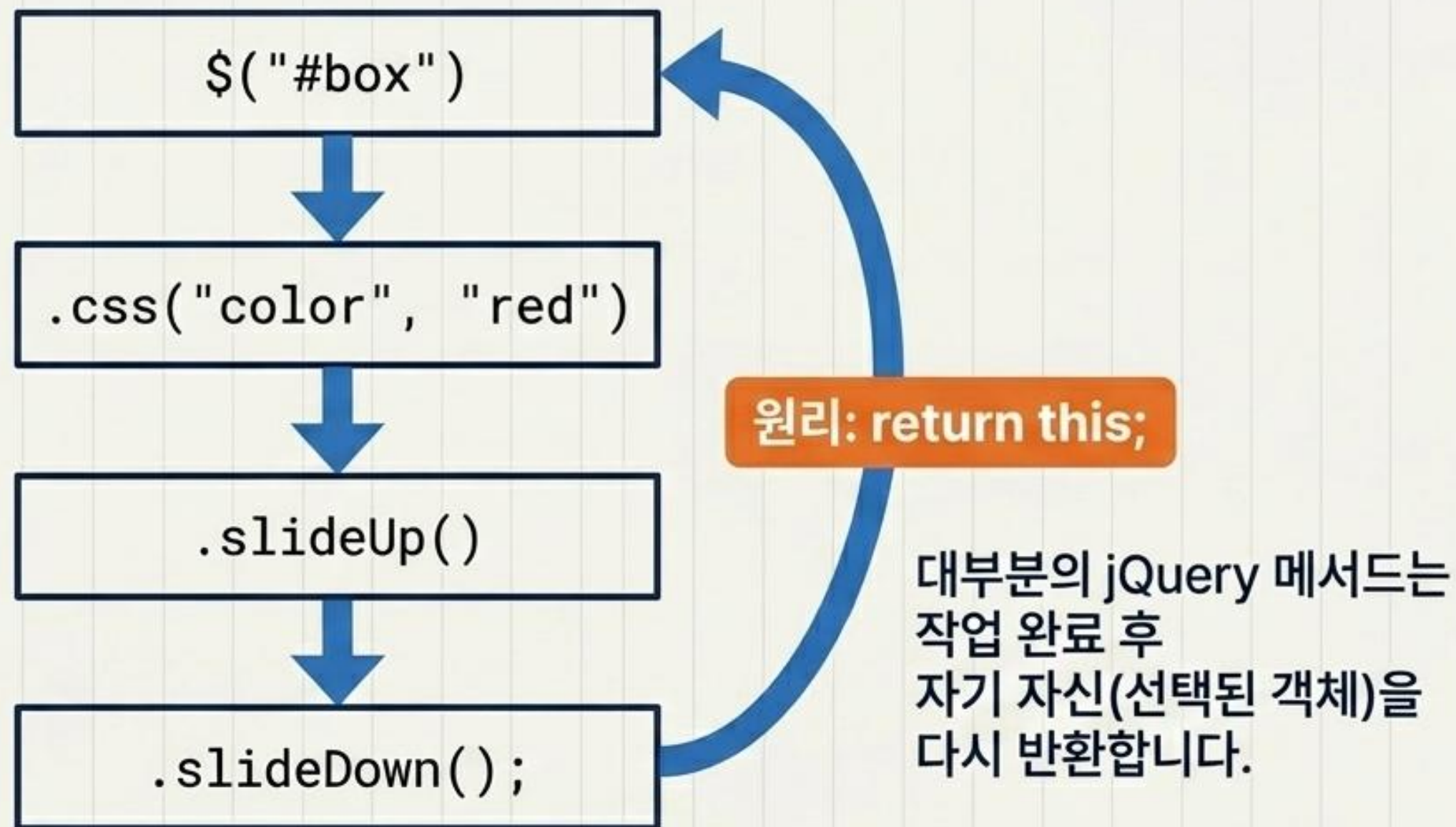


Custom Animation (자율 모션)

```
$("#box").animate({  
  left: '250px',  
  opacity: 0.5  
}, 1000);
```

→ CSS 속성과 지속 시간(ms)을 수치화하여 정밀하게 제어

| 마법의 파이프라인: 메서드 체이닝 (Method Chaining)



단 한 번의 DOM 탐색으로 여러 작업을 물 흐르듯 연속 수행하여 코드 가독성과 성능을 극대화합니다.

| 비동기 통신의 혁명 (The AJAX Revolution)

실무 예시: 화면 전체를 새로고침하지 않고,
실시간으로 댓글 목록만 업데이트!



Client
(웹 브라우저)

```
$.ajax({ ... }) {  
  $.get() / $.post()  
}
```

payload
method: GET/POST

Request (요청)

데이터 반환 (주로 JSON)

Response (응답)

Success Callback

```
success: function(res) {  
  $("#comments").html(res);  
}
```



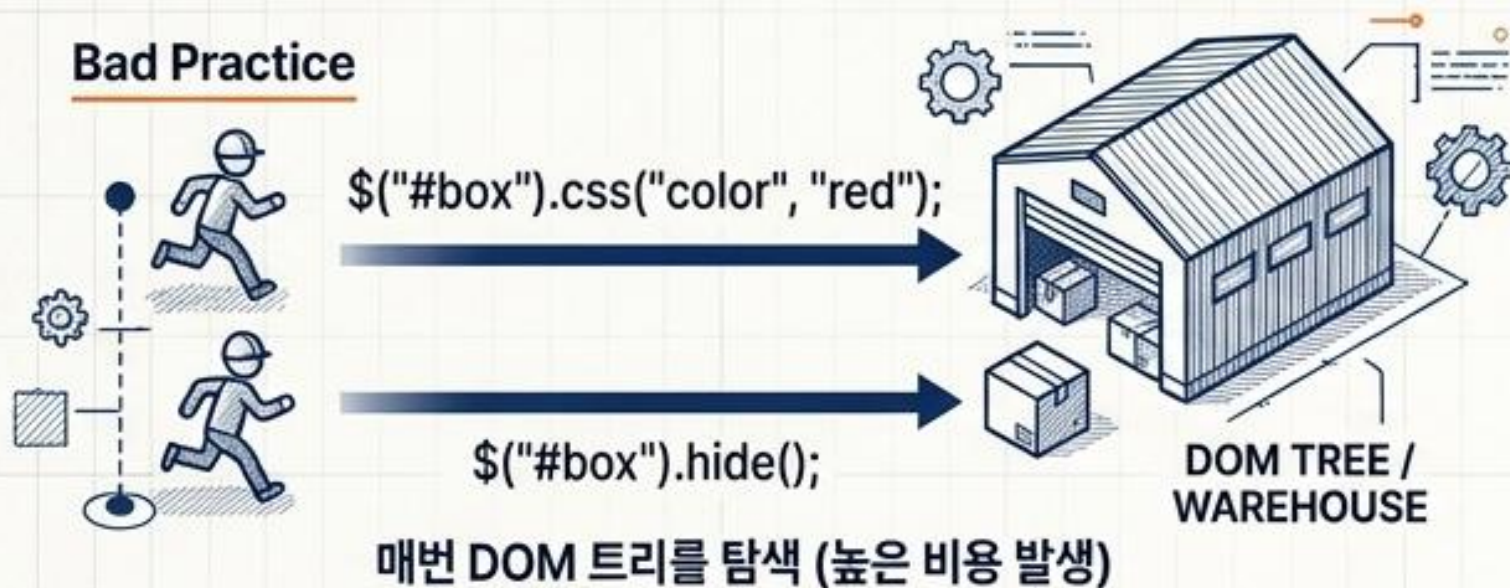
Server
(데이터베이스)

```
$.ajax({ ... }) {  
  ...  
}
```

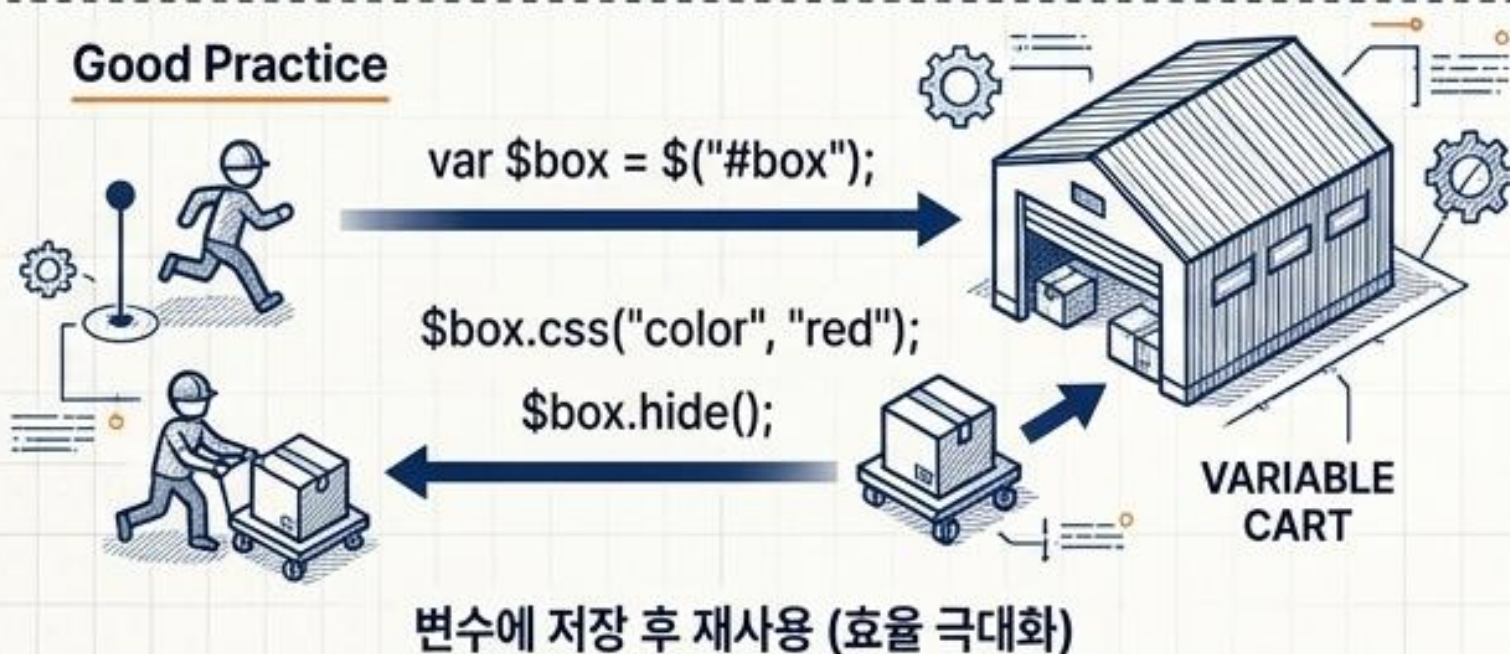

| 성능 최적화와 실무 패턴 (Best Practices)

선택자 캐싱 (Caching)

Bad Practice



Good Practice



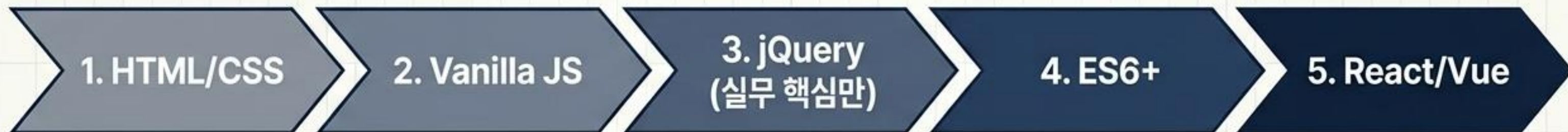
실무 필수 패턴: 폼 검증과 기본 동작 차단



```
$("#form").submit(function(e){  
  if($("#name").val() == "") {  
    alert("이름 입력");  
    e.preventDefault(); // 폼 전송(새로고침) 차단  
  }  
});
```


| 현대 웹 개발 환경에서의 jQuery 생태계

적극 사용 권장 (Use It)	사용 지양 (Skip It)
✓ 워드프레스 테마 개발 및 유지보수 ✓ 기존 SI 레거시 시스템 운영 ✓ 관리자 페이지 및 백오피스 구축 ✓ 빠른 UI 프로토타이핑	⊗ 최신 SPA 기반 프로젝트 ⊗ 대규모 상태 관리가 필요한 프론트엔드 ⊗ React / Vue / Next.js 도입 환경



결론: jQuery는 웹 프론트엔드 역사의 위대한 이정표이자, 실무 유지보수를 위한 가장 강력하고 필수적인 무기입니다.